Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

AeroDrop

Autonomous Indoor Drone for Search and Rescue

Tamer Zraiq (G00425053)

Project Engineering - Supervisor: Paul Lennon

Year 4

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025/2026

Project Graphic

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Tamer Zraiq

Acknowledgements

I would like to sincerely thank my supervisor, Paul, for his continuous support, guidance, and encouragement throughout the duration of this project.

I am especially grateful to Dan Moss from Manna Air Delivery for sharing his industry expertise and providing valuable technical insight that helped shape key aspects of the project.

I would also like to thank my classmate Pero for offering helpful input in several areas along the way, and Ameen Ghosheh for his graphical expertise and input which was used throughout the design of the poster.

AI language models, specifically Claude [31] and ChatGPT [32], were consulted throughout this project for debugging, code structure suggestions, and technical writing refinement. Their use is acknowledged here in the interest of academic transparency.

Finally, I would like to acknowledge the university staff for their support, resources, and assistance throughout my studies, which made this project possible.

Table of Contents

1 Summary

AeroDrop is a custom-built autonomous quadcopter designed to operate entirely without GPS. It targets search-and-rescue operations in indoor environments where satellite positioning is unavailable, mapping its surroundings in real time, detecting targets using onboard AI, and executing pre-planned missions under full software control with no RC transmitter involved at any stage.

The hardware platform is built around a TBS Source One V5 5-inch carbon fibre frame carrying a Matek H7A3-Slim flight controller running ArduCopter 4.6.3. Position hold in the horizontal plane is achieved through a Matek 3901-L0X sensor module combining a PMW3901 optical flow sensor and a VL53L0X time-of-flight rangefinder, both communicating with the flight controller over a single MSP serial link on SERIAL3. A Raspberry Pi 4 companion computer connects to the flight controller via a dedicated hardware UART at 115200 baud and runs the entire autonomy stack. An LD06 spinning 2D LiDAR on a separate Raspberry Pi UART at 230400 baud delivers 360-degree environmental scans. A Sony IMX500 AI camera runs object detection directly on the sensor's onboard neural processor, streaming inference results to the companion computer with zero host CPU load for inference.

The software is a layered autonomy stack. MAVProxy bridges the MAVLink stream from the flight controller to two UDP endpoints, allowing MAVSDK-Python and pymavlink to co-exist as simultaneous consumers. A seven-state finite state machine encodes valid flight phases with strict transition guards. An offboard executor sends 10Hz NED velocity setpoints to ArduCopter's GUIDED mode. A task executor generates boustrophedon scan patterns with configurable row length, row count, and altitude, monitoring the camera pipeline for detections in parallel. A 20m x 20m occupancy grid built from Bresenham ray-cast LiDAR scans feeds the mission map. A FastAPI backend serves a React dashboard over WebSocket, providing live telemetry, LiDAR visualization, occupancy map, AI detection events, and full mission control. Detections are logged with NED coordinates and JPEG frames, exported in a PDF mission report at mission end.

Confirmed results on real hardware: optical flow sensor live with FLOW_QUALITY=43, MAVLink telemetry pipeline end-to-end at 5Hz, LiDAR parsing 104,435 measurement points across real test sessions, IMX500 detecting objects at 62-81% confidence on the live dashboard. A fault initially pointing to the S3 motor output was isolated through a systematic ESC swap diagnostic as a failed ESC3 unit. A replacement ESC was purchased and installed, and all four motors are now confirmed spinning under power with propellers attached. The battery was depleted before a full takeoff test could be completed and is unable to charge due to the battery reaching max discharge, making it unusable. The takeoff implementation uses MAVSDK's native action.takeoff() sequence after confirming EKF local position validity, and the full autonomy stack is staged and ready for the first free-flight trial. A servo gripper integrated via pigpio hardware PWM on GPIO 12 provides the payload delivery mechanism, with auto-trigger on target detection confirmed working end-to-end in ground-test mode.

2 Terms & Concepts

The following table defines key terms and acronyms used throughout this report. These concepts are central to understanding the design and operation of the AeroDrop system.

Term / Acronym	Definition and Primary Source
MAVLink	Micro Air Vehicle Link. A lightweight binary serialisation protocol for unmanned vehicle communication, defining message types for telemetry, commands, and parameter access [6].
EKF3	Extended Kalman Filter version 3. ArduCopter's navigation core, which maintains a probabilistic state estimate by fusing data from multiple sensors at high frequency [2].
NED	North-East-Down. The local coordinate frame used for drone navigation. North is the positive X axis, East is positive Y, and Down is positive Z. Altitude is expressed as negative Down.
Optical Flow	A sensor technique that measures apparent image motion between successive camera frames to estimate translational velocity, used here to provide horizontal velocity to EKF3 without GPS [3].
ToF (Time-of-Flight)	A distance measurement method using the travel time of light pulses. The VL53L0X sends 940nm laser pulses and measures the reflected return time to compute altitude [12].
MSP	MultiWii Serial Protocol. A binary protocol originally developed for FPV flight controllers, used here for communication between the Matek 3901-L0X sensor and the flight controller [35].
SITL	Software In The Loop. A development environment where flight controller firmware runs as a normal process on a desktop, connected to a physics simulation, allowing full software testing without hardware [16].
GUIDED mode	An ArduCopter flight mode in which the vehicle accepts external position or velocity setpoints over MAVLink, required for autonomous waypoint following from the companion computer [1].
MAVSDK	A high-level SDK providing an async Python API for MAVLink flight commands (arm, takeoff, land, RTL) and telemetry subscriptions [7].

pymavlink	A low-level Python library providing direct MAVLink message-level access, used where MAVSDK does not expose the required message type [6].
MAVProxy	A MAVLink proxy and command-line ground station that holds the serial port exclusively and re-publishes the MAVLink stream to multiple UDP endpoints simultaneously [15].
FSM	Finite State Machine. A computational model that defines a system as a set of discrete states, with explicit rules governing which transitions between states are valid.
Boustrophedon	A back-and-forth scan pattern where each row is traversed in the opposite direction to the previous, minimising travel between rows [36].
Bresenham's Algorithm	An integer line-drawing algorithm used here to trace each LiDAR ray on the occupancy grid, marking every cell along the path as free and the endpoint as occupied [14].
BEC	Battery Eliminator Circuit. A DC-DC converter that produces a regulated lower voltage from the main battery for powering electronics.
UBEC	Universal BEC. An external standalone BEC module, used here as a replacement for the failed onboard PDB BEC.
PDB	Power Distribution Board. Distributes battery power to ESCs and provides regulated voltage outputs for flight electronics [28].
ESC	Electronic Speed Controller. Converts PWM throttle signals from the flight controller into three-phase motor drive signals for brushless motors.
UART	Universal Asynchronous Receiver-Transmitter. A hardware serial communication interface used here for the FC-to-RPi MAVLink link and LiDAR data.
CSI	Camera Serial Interface. The ribbon cable interface connecting the IMX500 camera module to the Raspberry Pi.
EKF Relative Aiding	An EKF3 operational mode where the vehicle maintains a local position estimate from optical flow and ToF without GPS, with position relative to the takeoff point rather than absolute global coordinates [2].

asyncio	Python's standard library for cooperative multitasking, used throughout the backend for non-blocking I/O, telemetry polling, WebSocket pushes, and mission execution [17].
FastAPI	A modern Python ASGI web framework used for the REST API and WebSocket endpoints serving the dashboard [11].
WebSocket	A full-duplex communication protocol over HTTP used for real-time telemetry, LiDAR point cloud, occupancy map, and detection event streaming to the dashboard [21].
ArduCopter	Open-source flight control firmware for multirotor vehicles, part of the ArduPilot project, running on the Matek H7A3-Slim at version 4.6.3 [1].

Table 2-1: Key Terms and Concepts

3 Poster

AERODROP

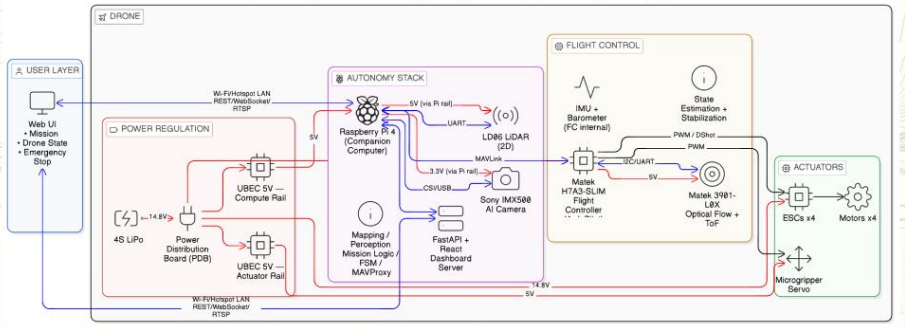
AN AUTONOMOUS DRONE FOR SEARCH & RESCUE

TAMER ZRAIQ
G00425053
4TH YEAR PROJECT

PROJECT OVERVIEW

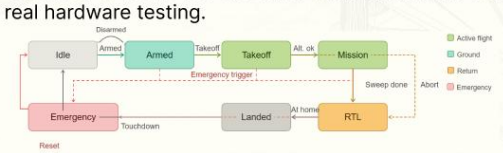
AeroDrop is a fully autonomous quadcopter built from scratch to navigate, map, and deliver payloads inside buildings where GPS does not work. Designed for search and rescue scenarios where ground robots cannot reach and conventional drones lose navigation, it operates without a pilot, without pre-mapped environments, and without any external positioning infrastructure. At its core is a custom-assembled drone running ArduCopter on a Matek flight controller, paired with a Raspberry Pi 4 that handles all mission logic, a 360° LiDAR for live spatial mapping, and an AI camera for target detection at the delivery zone.

ARCHITECTURE DIAGRAM



SOFTWARE

- 7-state Python FSM orchestrates full mission arming to landing driven by live MAVLink telemetry.
- LiDAR scans converted in real time to occupancy grid feeding A* path planner.
- Optical flow + ToF feed EKF3 for stable GPS-denied indoor positioning.
- FastAPI + React dashboard streams telemetry, FSM state, occupancy map, and live video over local hotspot.
- IMX500 AI camera performs on-sensor delivery-zone detection.
- Validated in ArduPilot SITL and HITL simulation before real hardware testing.



HARDWARE

- 4S LiPo → PDB → dual isolated UBEC 5V rails.
- 3901-L0X optical flow + ToF → FC via I2C, fused via EKF3 for GPS-denied position hold.
- LD06 LiDAR → Pi via UART (230400 baud) → occupancy grid → A* planner.
- Pi ↔ FC via UART (115200 baud), pymavlink, commands down, telemetry up.
- IMX500 on-sensor inference → results to Pi via CSI, no frame streaming.
- FC → 4x ESCs via PWM + microgripper servo on actuator rail.

RESULTS

Built and soldered from scratch, with the full autonomy stack written independently. Validated in ArduPilot SITL and HITL simulation prior to real hardware testing. Stable GPS-denied position hold confirmed, all sensors live, and the end-to-end SAR pipeline runs from arming to payload delivery with real-time dashboard monitoring.

Scan the QR code to view the full results with images and documentation.

RESULTS

TOOLS & TECH

Websocket	FastAPI	Raspberry Pi OS
ArduCopter 4.6.3	React, Vite	A* Pathfinding
Edge AI Inference	EKF3	ArduPilot SITL, HITL
MAVLink/ pymavlink	MAVSDK	Point Cloud Processing

ATU

BENG SOFTWARE & ELECTRONICS ENGINEERING

4 Introduction

Indoor autonomous flight remains an unsolved problem in applied robotics. While GPS-guided drones operate reliably outdoors, entering a building immediately removes the navigation stack's primary input. AeroDrop addresses this directly: a quadcopter platform that navigates, maps, and delivers payloads without any satellite signal.

The motivation is practical. Search-and-rescue teams operating in collapsed structures, warehouse automation tracking inventory across floors, and last-mile delivery in multi-story interiors all require vehicles that work without GPS. These are domains where human operators face significant risk and where autonomous systems offer meaningful capability.

AeroDrop is implemented as a full-stack embedded systems project. At the hardware level it is a custom-assembled quadcopter using a dedicated flight controller, optical flow sensor, LiDAR, and companion computer. At the software level it combines real-time MAVLink control, a Python finite state machine, a REST and WebSocket backend, and a React ground control dashboard. Every control action is generated programmatically by software on the Raspberry Pi, transmitted over a serial MAVLink link, with no RC transmitter in the loop.

Section 5 provides technical background on GPS-denied navigation, the relevant firmware and protocols, and sensor operation. Section 6 describes the system architecture. Section 7 documents the project plan across both semesters. Sections 8 and 9 cover hardware and firmware configuration. Section 10 covers the software architecture. Section 11 provides an annotated code walkthrough. Section 12 covers testing and results. Section 13 documents the key engineering challenges encountered and how each was resolved. Section 14 addresses ethics. Section 15 concludes.

5 Background

5.1 Project Origins and Design Rationale

The project began in May 2025 as AutoCopter, a domestic assistant drone using a Navio2 flight controller hat on the Raspberry Pi with voice control, object detection, and a servo gripper. The core problem was immediately evident: the concept solved no real problem. The pivot to SAR and industrial inspection in June/July 2025 gave the project genuine purpose, targeting environments where autonomous aerial vehicles directly reduce human risk.

Two subsequent concept iterations were evaluated and rejected. A drone swarm approach involving multiple vehicles with heterogeneous roles and mesh networking was compelling technically but required three or more physical vehicles and was impractical within cost and time constraints. A hybrid drone-rover approach was also considered before being dropped. The final architecture settled on a single GPS-denied indoor drone with a full autonomy stack.

Two architectural decisions made early had lasting consequences. First, the flight controller was changed from Navio2 (a Raspberry Pi HAT) to the Matek H7A3-Slim (a dedicated flight controller). The Navio2 shares the Raspberry Pi's ARM Cortex-A72 processor, creating a fundamental conflict: flight control requires real-time deterministic computation at 400Hz while the autonomy stack (asyncio event loop, LiDAR parsing, WebSocket streaming, camera subprocess) is non-deterministic and CPU-intensive [33]. Running both on the same processor means one starves the other. The H7A3-Slim's dedicated STM32H7A3 Cortex-M7 [33] handles all real-time flight computation independently, leaving the Raspberry Pi entirely free for autonomy software. Second, SITL development was migrated from PX4 to ArduCopter after industry advisor Dan Moss identified that PX4 and ArduCopter differ in parameter names, command handling, and some message semantics, meaning every command tested in PX4 SITL would need re-testing on real ArduCopter hardware, defeating the purpose of simulation.

5.2 GPS-Denied Navigation

Global Navigation Satellite Systems (GNSS) provide outdoor position estimates accurate to within a few metres under good conditions. Indoors, multipath reflections, signal attenuation through

structural materials, and complete signal blockage make GNSS-based positioning unreliable or unavailable. Alternative localisation approaches are required. AeroDrop uses optical flow as the primary horizontal velocity source, fused through ArduCopter's EKF3 filter with time-of-flight altitude data and IMU measurements. The LD06 LiDAR provides supplementary 2D range data for obstacle awareness and map construction, as described in Section 10.6.

5.3 ArduCopter and EKF3

ArduCopter is open-source flight control firmware for multirotor vehicles [1]. It implements a hierarchical control architecture: an outer position and velocity loop feeds setpoints to an attitude controller, which outputs motor mixing commands at high frequency. The Extended Kalman Filter (EKF3) is the navigation core, maintaining a probabilistic state estimate by fusing data from multiple sensors [2]. The prediction step runs at IMU rate (approximately 400Hz) and the correction step runs when sensor measurements arrive at their respective rates, as described by Welch and Bishop in their introduction to the Kalman filter [30].

For GPS-denied operation, EKF3 must be configured to accept optical flow as the primary horizontal velocity source. ArduCopter exposes this configuration through a flat key-value parameter store of approximately 1,200 parameters[1]. The full parameter table is in Section 9.1. A critical discovery during development: ARMING_CHECK=0 does not bypass EKF3 pre-arm checks. The filter maintains its own internal health state through a completely separate code path and will refuse to allow arming regardless of the software arming check parameter. The full EKF3 arming dependency chain is described in Section 9.2.

5.4 MAVLink Protocol

MAVLink is a lightweight binary serialization protocol for unmanned vehicle communication [6]. Version 2, used throughout this project, specifies a packet structure containing a start byte (0xFD), payload length, incompatibility and compatibility flags, a packet sequence number, system ID, component ID, a three-byte message ID, payload, and a two-byte CRC seeded with a per-message-ID value [6].

Figure 5-1 MAVLink V2 Packet [6]

For example, the HEARTBEAT message (ID 0) is exchanged at 1Hz between all MAVLink participants and carries system type, autopilot type, base mode flags, custom mode, and system state. If the flight controller stops receiving HEARTBEAT from a connected system, it can trigger a failsafe, which is why MAVProxy and MAVSDK must both be running for the system to remain armed.

Two Python libraries interface with MAVLink in this project for different purposes. MAVSDK-Python provides a high-level async API for flight commands and telemetry subscriptions, handling connection management and heartbeat maintenance automatically [7]. pymavlink provides raw message-level access without abstraction [6]. pymavlink is used specifically for 10Hz NED position setpoints via SET_POSITION_TARGET_LOCAL_NED because MAVSDK's offboard API targets PX4's offboard mode, which does not exist in ArduCopter. The practical consequence of this is examined in Section 10.4.

5.5 Optical Flow Sensing and the Altitude Dependency

The Matek 3901-L0X integrates a PMW3901 optical flow sensor with a VL53L0X time-of-flight distance sensor [4]. The PMW3901 captures 35x35 pixel images of the surface below at up to 400 frames per second and computes frame-to-frame displacement using an onboard correlation ASIC, outputting a signed pixel displacement vector and a quality metric (SQUAL, 0-255) per frame [13]. The VL53L0X measures altitude using 940nm VCSEL laser pulses, with a reliable operating range of approximately 30mm to 2000mm [12].

There is a hard mathematical dependency between the two sensors. ArduCopter converts pixel displacement to body-frame velocity using the formula [3]:

$$velocity = \frac{pixel\_displacement * altitude}{focal\_length * dt}$$

If altitude is zero or out of range, the result is undefined and EKF3 rejects the measurement. This means optical flow and rangefinder altitude cannot function independently; both must be healthy simultaneously before EKF3 accepts any velocity data. The full chain of conditions this creates is detailed in Section 9.2.

6 Project Architecture

AeroDrop is structured as three communicating layers: the firmware layer (Matek H7A3-Slim running ArduCopter 4.6.3 with its sensors), the companion computer layer (Raspberry Pi 4 running the full autonomy stack), and the ground control layer (any network-connected device accessing the React dashboard). Each layer has a precisely defined interface. Figure 6-1 shows the system architecture.

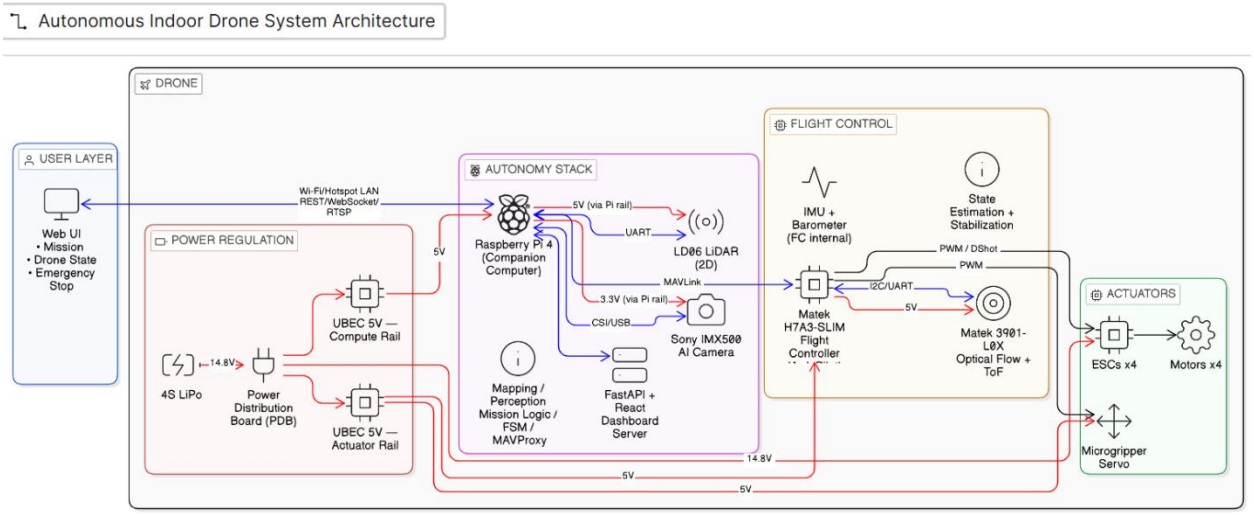


Figure 6-1 Architecture Diagram

6.1 Communication Topology

MAVLink is the central communication bus. The flight controller's SERIAL2 port connects to the Raspberry Pi's /dev/ttyAMA2 UART at 115200 baud, enabled by the dtoverlay=uart2 device tree overlay in /boot/firmware/config.txt. A single process can hold a serial port exclusively, so a separate bridging process is required to allow multiple consumers of the MAVLink stream. MAVProxy handles this by reading /dev/ttyAMA2 and re-publishing the stream to two UDP endpoints simultaneously: port 14552 consumed by MAVSDK-Python, and port 14553 consumed by pymavlink [15]. Neither control library touches the serial port directly. This single-reader, multiple-consumer model prevents UART contention while allowing co-existing telemetry and command consumers.

The optical flow sensor communicates directly with the flight controller over SERIAL3 using MSP. The LiDAR communicates directly with the Raspberry Pi over /dev/ttyAMA0, invisible to the flight controller. The IMX500 communicates with the Raspberry Pi over the CSI camera interface. These

three sensor channels are completely independent of each other and of the MAVLink bus. Table 6-1 summarizes all communication links.

FC to Raspberry Pi	**SERIAL2 / /dev/ttyAMA2 / MAVLink v2 / 115200 baud**
RPi to MAVSDK-Python	UDP 14552 (MAVProxy forward)
RPi to Ground Station	UDP 14553 (MAVProxy forward)
LiDAR to RPi	/dev/ttyAMA0 / 230400 baud / LD06 binary protocol
Optical Flow to FC	SERIAL3 / MSP / FLOW_TYPE=7
Dashboard to Backend	WebSocket ws://172.20.10.2:8000/ws
User to Dashboard	HTTP, React SPA served by FastAPI

Table 6-1: AeroDrop Communication Links

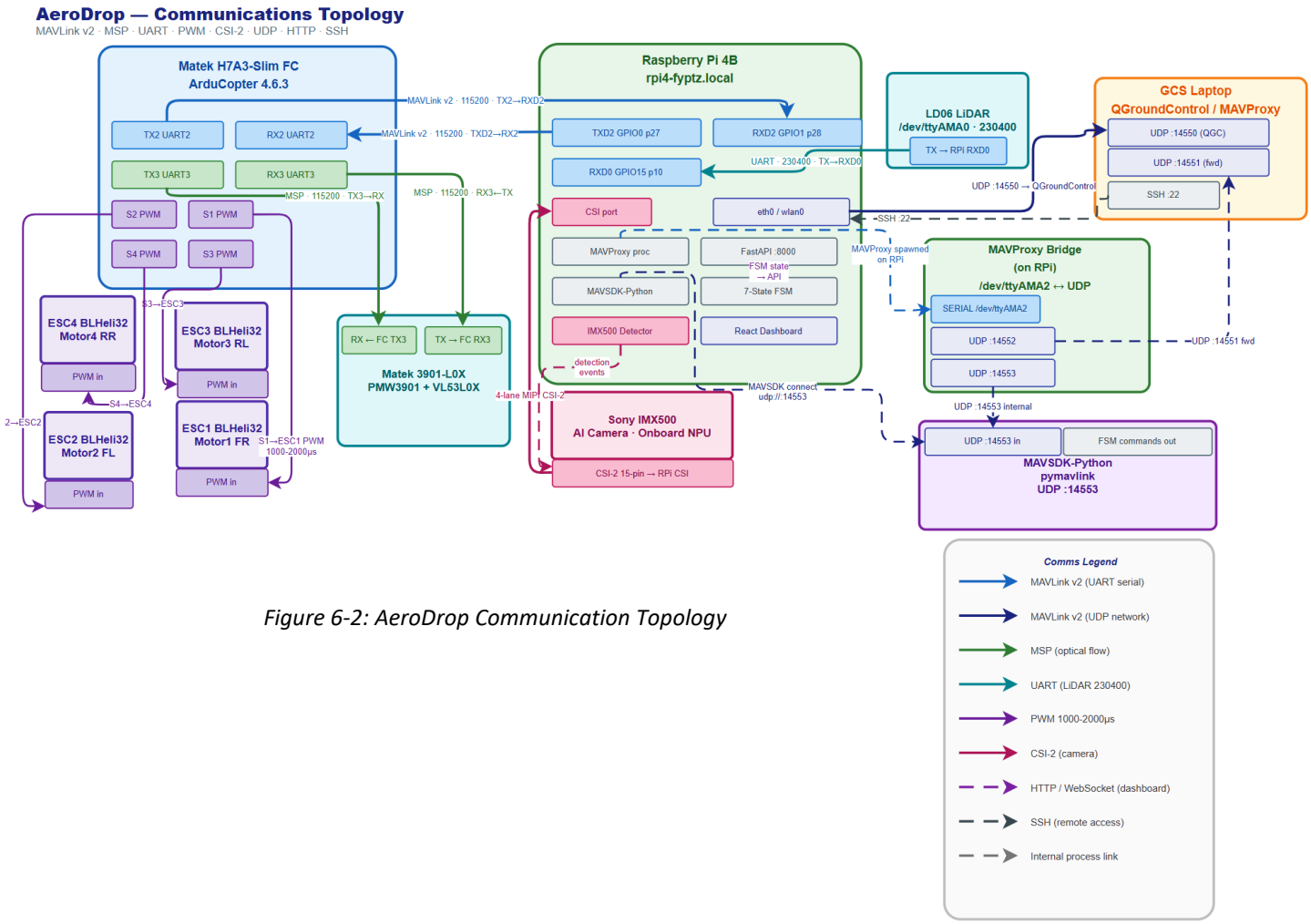


Figure 6-2: AeroDrop Communication Topology

6.2 Concurrency Architecture

The system is not a collection of independent sequential modules. It is a concurrent system with multiple simultaneous execution contexts on one Raspberry Pi 4, all sharing the same hardware and operating on common data structures. Two distinct concurrency primitives are used. The asyncio event loop [17] runs all flight-critical coroutines cooperatively, yielding control at await points and protected by asyncio.Lock for shared state. Separately, the LiDAR parser and camera handler run in daemon threads because they perform blocking I/O. serial.read() blocks, and camera frame capture blocks. Blocking inside an asyncio coroutine would freeze the entire event loop, so daemon threads with threading.Lock are the correct structure for these. The two primitives are not interchangeable: threading.Lock inside an async coroutine will deadlock the event loop, and asyncio.Lock inside a daemon thread has no event loop context to acquire. Table 6-2 lists all 18 simultaneous execution contexts.

Execution Context	Type / Rate / Purpose
MAVProxy	OS process, continuous. Serial port exclusive holder, re-publishes MAVLink stream to UDP 14552 and 14553.
uvicorn/FastAPI	asyncio event loop, continuous. HTTP and WebSocket server.
_poll_connection	asyncio coroutine, event-driven. MAVSDK connection_state() subscription.
_poll_armed	asyncio coroutine, event-driven. MAVSDK telemetry.armed() subscription.
_poll_flight_mode	asyncio coroutine, event-driven. MAVSDK telemetry.flight_mode() subscription.
_poll_position	asyncio coroutine, event-driven. MAVSDK telemetry.position(), lat, lon, rel_alt.
_poll_battery	asyncio coroutine, event-driven. MAVSDK telemetry.battery(), filters sentinel values.
_poll_local_position	asyncio coroutine, event-driven. MAVSDK position_velocity_ned(), NED coordinates.
_poll_heading	asyncio coroutine, event-driven. MAVSDK attitude_euler(), yaw heading.

LidarStreamer._run	daemon thread, ~10Hz. Serial read /dev/ttyAMA0, decode LD06 packets, fill ring buffer.
Picamera2 capture loop	daemon thread, 15fps. IMX500 frame capture, inference result extraction, JPEG encode.
ServoGripper.drop background task	asyncio Task, event-driven. Spawned by detection monitor or API endpoint. Open -> 2s hold -> close. Non-reentrant via asyncio.Lock.
/ws/telemetry	asyncio coroutine, 5Hz. TelemetrySnapshot to JSON to all WebSocket clients.
/ws/map	asyncio coroutine, 2Hz. OccupancyGrid.serialise(), 40,000 cells to WebSocket.
/ws/lidar	asyncio coroutine, 10Hz. LidarStreamer.snapshot(), 1,500 points to WebSocket.
OffboardExecutor.run	asyncio coroutine, 10Hz. NED setpoints via pymavlink.
_monitor_detections	asyncio coroutine, 5Hz. Camera detections stamped with NED position, logged to DetectionLog.
_update_map	asyncio coroutine, 5Hz. LiDAR snapshot fed to OccupancyGrid.update().
_watch_mission_end	asyncio coroutine, 0.5Hz. Auto-RTL on sweep completion.

Table 6-2: Concurrent Execution Contexts

6.3 TelemetrySnapshot: Shared State Contract

TelemetrySnapshot is the single shared data structure connecting the flight controller state to every module that needs it: the WebSocket push, the offboard executor's arrival check, the task executor's NED logging, and the occupancy grid update. Seven asyncio coroutines write to it. Reads return a shallow copy via snapshot(), so a reader can never see a partially-written state while a polling coroutine is mid-update. Writes are protected by asyncio.Lock inside each _poll_* coroutine. This design is examined with code in Section 11.1.

7 Project Plan

The project spanned two academic semesters, each with a distinctly different character. Semester 1 was primarily conceptual and preparatory: ideas were explored, discarded, and refined; components were researched and ordered; the software stack was built and validated in simulation. Semester 2 was physical and integrative: real hardware was assembled, firmware was configured, sensors were brought up one by one, and the full pipeline was progressively validated under real conditions. Figure 7-1 shows the overall actual project timeline.

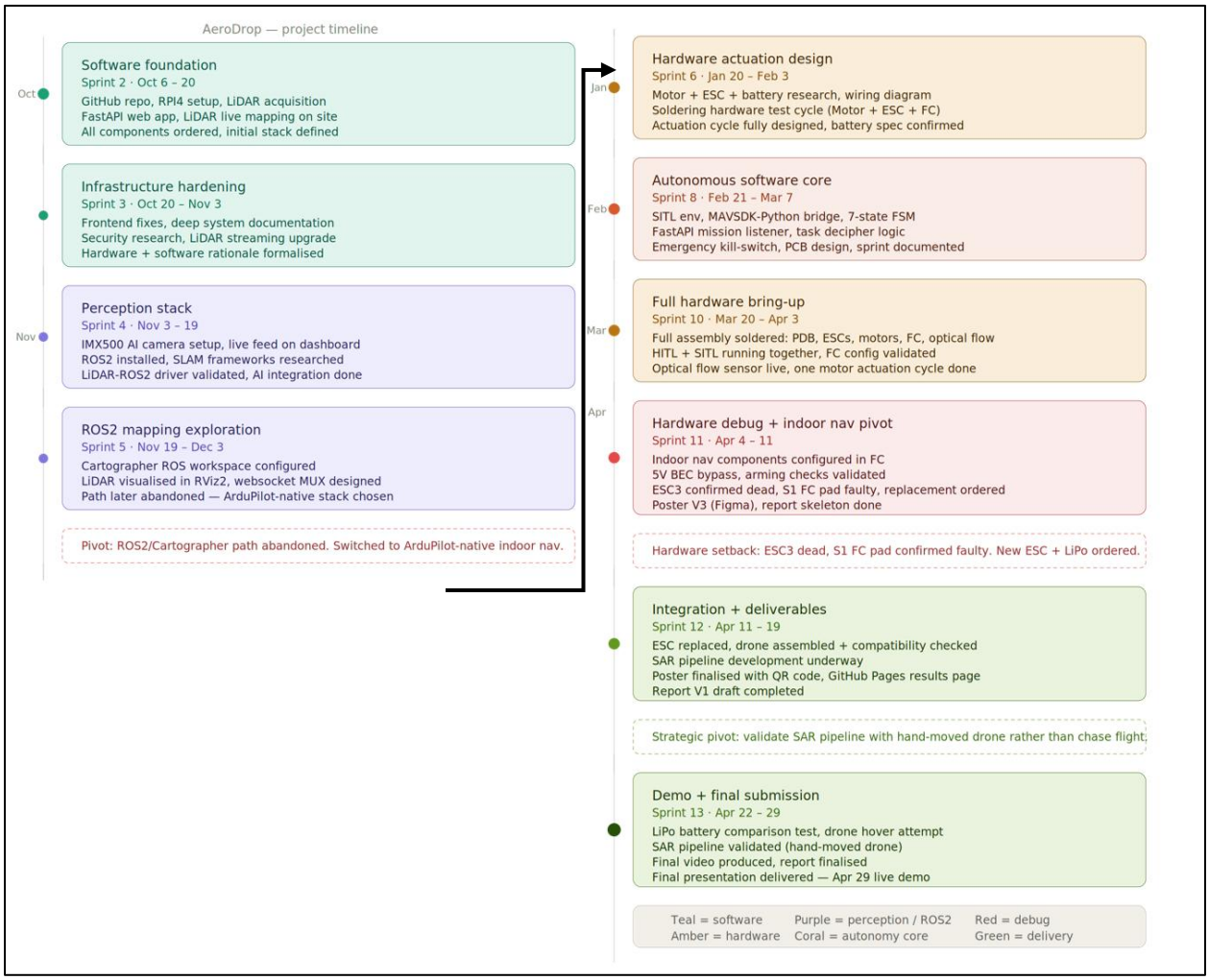


Figure 7-1: Project Timeline

7.1 Semester 1 (October 2025 to January 2026)

Semester 1 began with the concept exploration documented in Section 5.1. The AutoCopter domestic assistant concept was evaluated and rejected. Drone swarm and drone-rover hybrid

approaches were considered and discarded. By October/November 2025 the scope had settled on a single GPS-denied indoor SAR drone.

Hardware was researched and ordered during this period. The flight controller choice moved from Navio2 to the Matek H7A3-Slim based on the CPU contention argument described in Section 5.1. The Sony IMX500 was the first hardware element to be fully validated: setting it up required reflashing the Raspberry Pi OS from Trixie to Bookworm because the Trixie rpicam-apps binary was compiled without the ENABLE_POST_PROCESSING=1 CMake flag, meaning the IMX500's post-processing pipeline was absent entirely [9]. Once on Bookworm, live object detections were confirmed on the dashboard in November 2025: person at 81%, keyboard at 73%, and chair at 62% confidence.

The full software stack was developed and validated against ArduCopter SITL [16] in WSL2 during Semester 1. By the Christmas demo, the following modules existed and had been tested end-to-end in simulation: the seven-state FSM, DroneController with full telemetry polling, OffboardExecutor with the boustrophedon waypoint pattern, TaskExecutor with detection monitoring, OccupancyGrid with Bresenham ray-casting, the FastAPI backend with all REST and WebSocket endpoints, and the React dashboard with mission controls, live LiDAR canvas, occupancy map, and PDF export.

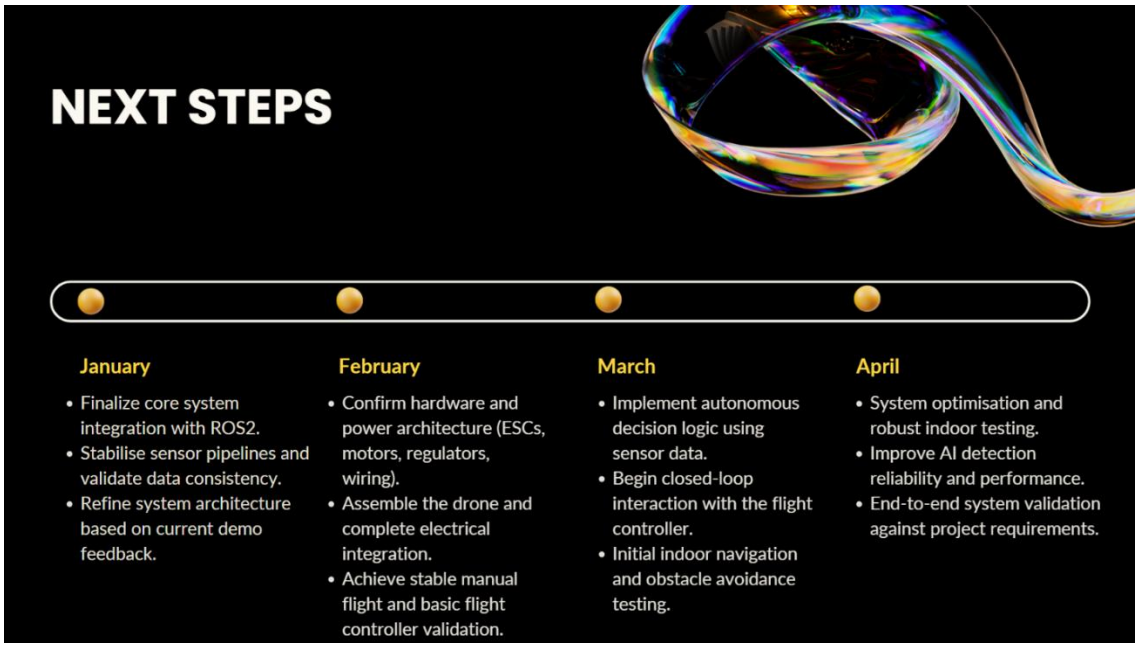


Figure 7-2: Christmas Demo planned next steps

7.2 Semester 2 (January 2026 to April 2026)

Semester 2 began with hardware assembly and physical bring-up. The plan from the Christmas demo, shown in Figure 7-2, identified the steps in the order: wire and verify the optical flow and ToF sensor, complete motor testing and ESC calibration, execute an ARM and hover test, validate the full SAR pipeline hand-held, and run an autonomous flight mission. The actual development path diverged significantly from this linear plan. Table 7-1 compares the planned steps against what actually happened, with references to the challenge documentation in Section 13.

Planned Step	What Actually Happened
Wire optical flow and ToF sensor	Completed, but RNGFND1_MAX_CM was found set to 1, causing EKF3 to reject every altitude reading. Fixed after systematic MAVProxy parameter investigation. See Section 13.2.
Motor testing and ESC calibration	Motor 3 failed to spin. Systematic swap diagnostic confirmed ESC3 was the faulty unit. Replacement purchased and installed. See Sections 8.6 and 13.1.
ARM and hover test	All 4 motors confirmed spinning with props under battery power. PDB/UBEC voltage sag caused RPi brownouts requiring separate USB-C power. RC failsafe (FS_THR_ENABLE) caused automatic disarm 2 seconds after arming. Both fixed. Battery depleted before full takeoff test. See Sections 13.1 and 13.2.
Full SAR pipeline validation	Full pipeline validated in SITL and partially hand-carried on real hardware. Blocked on LiDAR scan plane obstruction from loose internal wiring. See Section 13.3.
Autonomous flight mission	Pending. EKF3 relative aiding confirmed, GUIDED mode arm confirmed, takeoff code updated. First hover trial pending battery recharge.

Table 7-1: Planned vs Actual Development Progress in Semester 2

8 Hardware Design

8.1 System Overview and Component Selection

Hardware selection was constrained by four requirements: ArduCopter firmware compatibility, optical flow subsystem support at the firmware level, payload capacity sufficient for the Raspberry Pi 4 and all peripherals, and physical form factor for indoor operation. The TBS Source One V5 5-inch frame provides the structural foundation. Table 8-1 lists all principal components, and the complete bill of materials including suppliers and costs is in Appendix A.

Component	Specification
Frame	TBS Source One V5 5", full carbon fibre, ~124g [18].
Flight Controller	Matek H7A3-Slim [33], STM32H7A3 280MHz Cortex-M7, ArduCopter 4.6.3 [1].
Motors (x4)	EcoIII 2306 2400KV brushless [27], ~90g total.
ESCs (x3)	FVT LittleBee Summer 35A BLHeli32 [19,28], Rev 32.9, PWM 1000-2000us, ~60g total (for 4 ESCs).
ESC (ESC3)	Flash Hobby Arthur / DYS ARIA Slim 40A 3-6S BLHeli32 3-6S ESC.
Optical Flow + ToF	Matek 3901-L0X [4], PMW3901 [13] + VL53L0X [12], MSP on SERIAL3, ~4g.
LiDAR	LD06 [5], 360 degrees, 12m range, 230400 baud, /dev/ttyAMA0, ~97g.
Companion Computer	Raspberry Pi 4 Model B 4GB [10], Bookworm 64-bit, ~46g.
AI Camera	Sony IMX500 [8], onboard MobileNet-SSD [24], CSI, ~25-35g.
Battery	GNB 1850mAh 4S 100C XT60, ~170-185g.
PDB	Matek PDB-XT60, 5V and 12V BEC, ~15g.
UBEC	Voltage Regulator 2-13S, 5V 5A, replaces PDB BEC for FC power, ~15-20g.
Gripper	Pololu servo gripper, payload delivery, hardware fitted.
Estimated AUW	~686-730g (component estimates; scale measurement pending).

Table 6-1: AeroDrop Hardware Components

8.2 Flight Controller: Matek H7A3-Slim

The H7A3-Slim was selected for its STM32H7A3 processor running at 280MHz [33]. The H7 series offers approximately double the processing headroom of F4/F7 controllers, which

matters when EKF3 must simultaneously fuse optical flow, rangefinder, barometer, and IMU data while running the full attitude and position control loops. The board exposes six hardware UARTs. SERIAL2 (TX2/RX2 pads) is the companion computer MAVLink link, and SERIAL3 (TX3/RX3 pads) carries the optical flow MSP stream from the 3901-L0X. This multi-UART capability is a hardware requirement for this project, as the system needs simultaneous MAVLink output on one port and sensor input on another with no shared buffering.

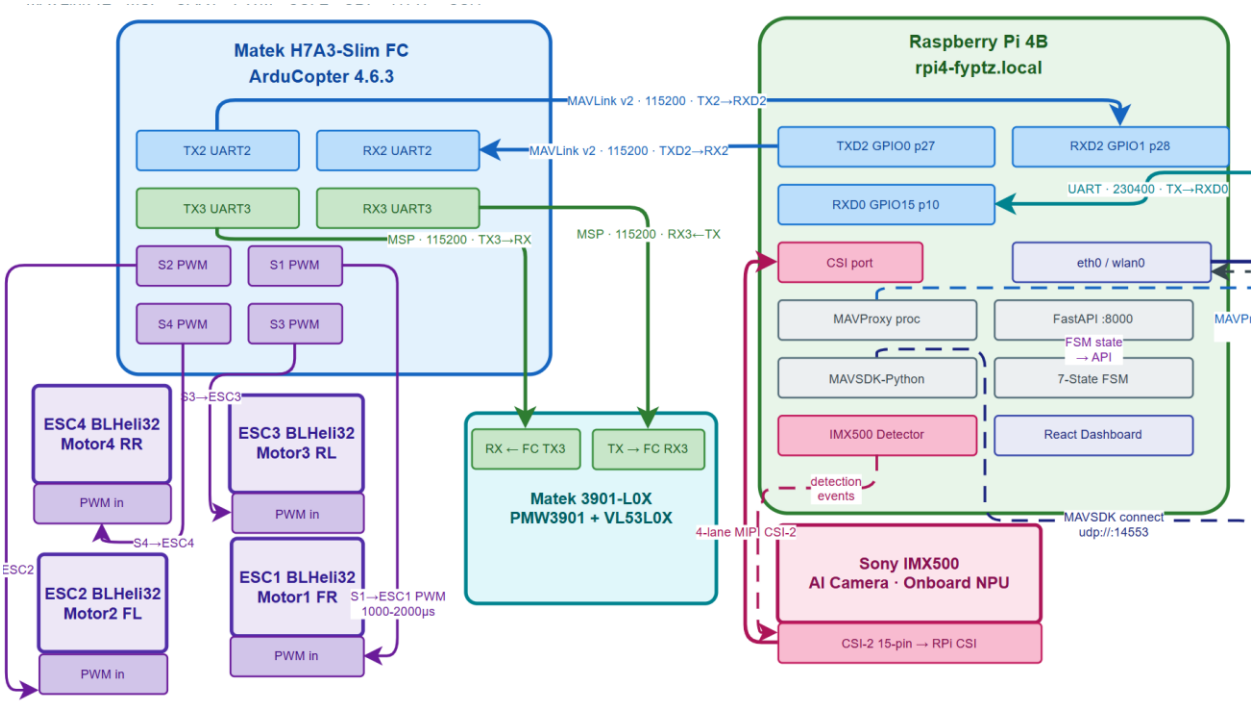


Figure 8-10: Flight Controller Communication Topology

A property discovered during hardware bring-up: the 5V BEC output pads on the H7A3-Slim are only active when a LiPo battery is connected. USB power alone delivers approximately 0.05V on the BEC pads. Any peripheral powered from the FC's 5V pads, including the Matek 3901-L0X, is completely unpowered during USB-only bench testing. This discovery changed every bring-up procedure: optical flow verification required both USB (for Mission Planner) and LiPo connected simultaneously, managed through a smoke stopper for short-circuit protection.

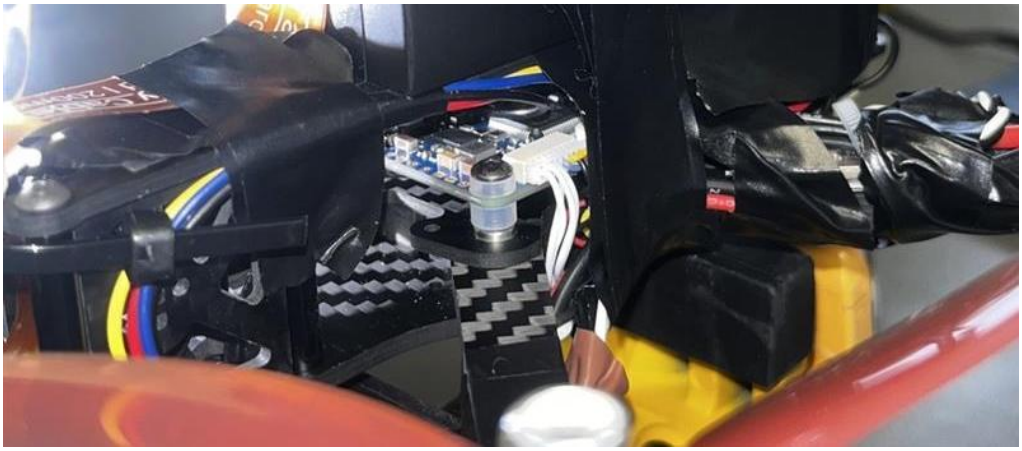


Figure 8-1: Matek H7A3-Slim flight controller on AeroDrop

8.3 Power Architecture and the PDB BEC Failure

The Matek PDB-XT60 distributes battery power to the four ESCs via copper power planes and provides regulated 5V and 12V outputs via onboard BEC circuits. In the original design, the PDB 5V BEC powered the flight controller's 5V pad. During motor bring-up testing, the PDB 5V BEC became unreliable, collapsing within a few seconds under any sustained load. An external Voltage Regulator Module (2-13S input, 5V 5A output) was wired directly to the PDB's raw battery voltage pads, with its output feeding the FC 5V pad.

A second, separate power problem emerged in Session 6 once all four motors were running simultaneously. The Raspberry Pi, powered via the UBEC connected to the PDB, experienced brownouts whenever the motors were armed and throttled up. The cause is voltage sag on the PDB bus under motor load: when four 2306 motors spin up, instantaneous current demand can exceed 20A, dragging the bus voltage below the UBEC's regulated output threshold. The temporary fix is powering the Raspberry Pi via USB-C from a separate source. The permanent fix for flight is a USB power bank mounted to the frame, providing a clean, motor-load-independent 5V supply. Figure 8-2 illustrates the power architecture with the UBEC bypass.

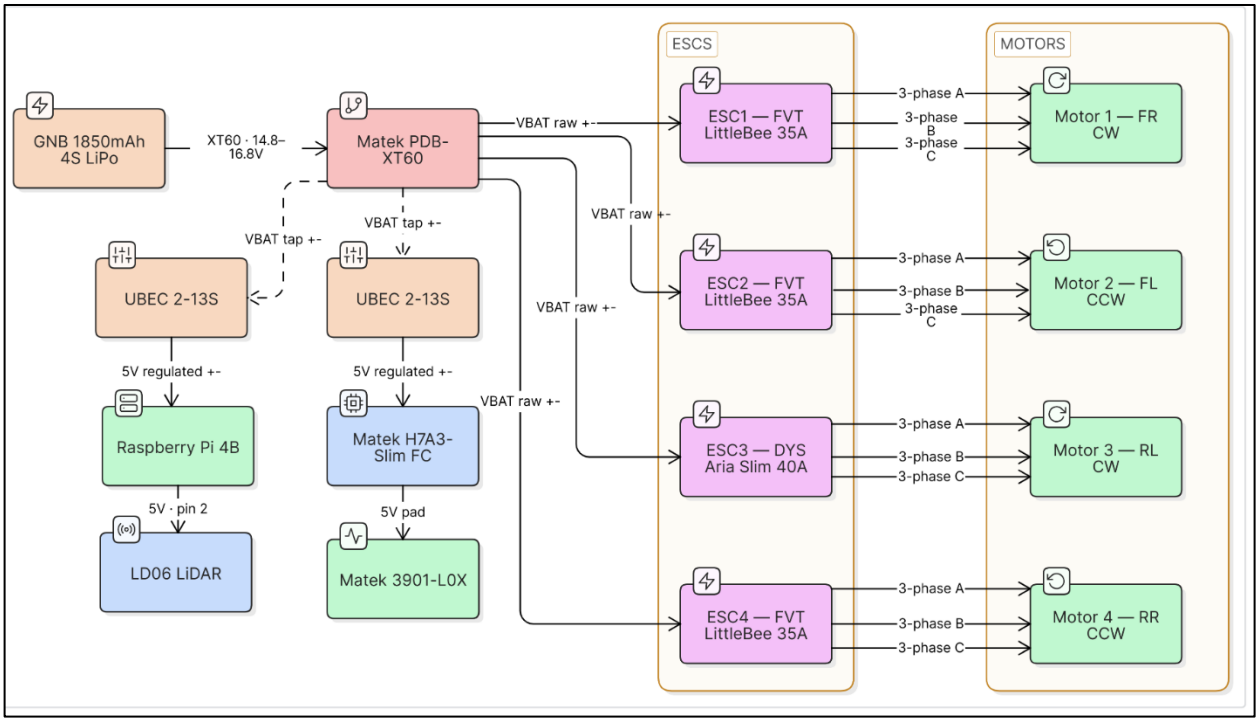


Figure 8-2: AeroDrop power architecture with UBEC bypass

8.4 Optical Flow and Altitude Sensing

The 3901-L0X packages three components: the PMW3901 optical flow ASIC, the VL53L0X time-of-flight sensor, and an STM32 microcontroller that reads both sensors and frames them in MSP packets for the flight controller [4]. As explained in Section 5.5, these two sensors are mathematically coupled through the altitude-to-velocity scaling formula and must both be healthy for EKF3 to accept flow measurements.

The sensor is confirmed live on real hardware. Mission Planner's Status tab shows opt_m_x = -0.00011, opt_m_y = 0.02050, and opt_qua = 43, seen in Figure 8-5. The quality value of 43 represents the number of valid surface feature correspondences the PMW3901's correlation ASIC found between consecutive frames [13], obtained by pointing the sensor at the lab floor at approximately 30cm range. It is sufficient for EKF3 acceptance, confirmed by the pre-arm health indicator transitioning to green after the mandatory convergence period described in Section 9.2.

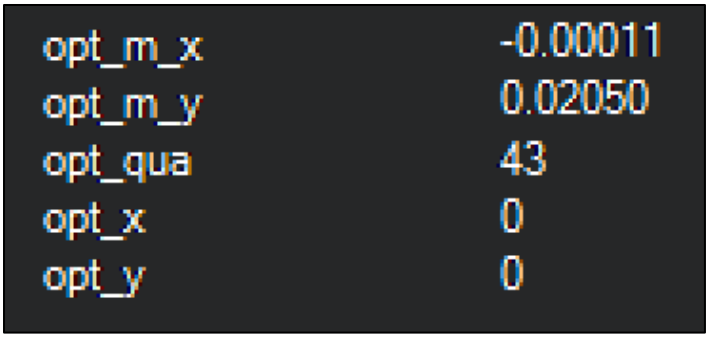


Figure 8-5: Screenshot of Optical Flow Data in Mission Planner

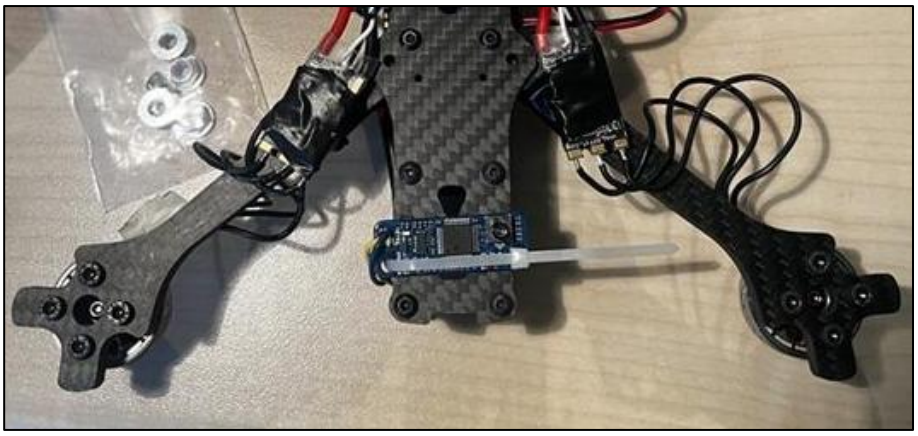


Figure 8-3: Matek 3901-L0X [4] mounted on upside down AeroDrop

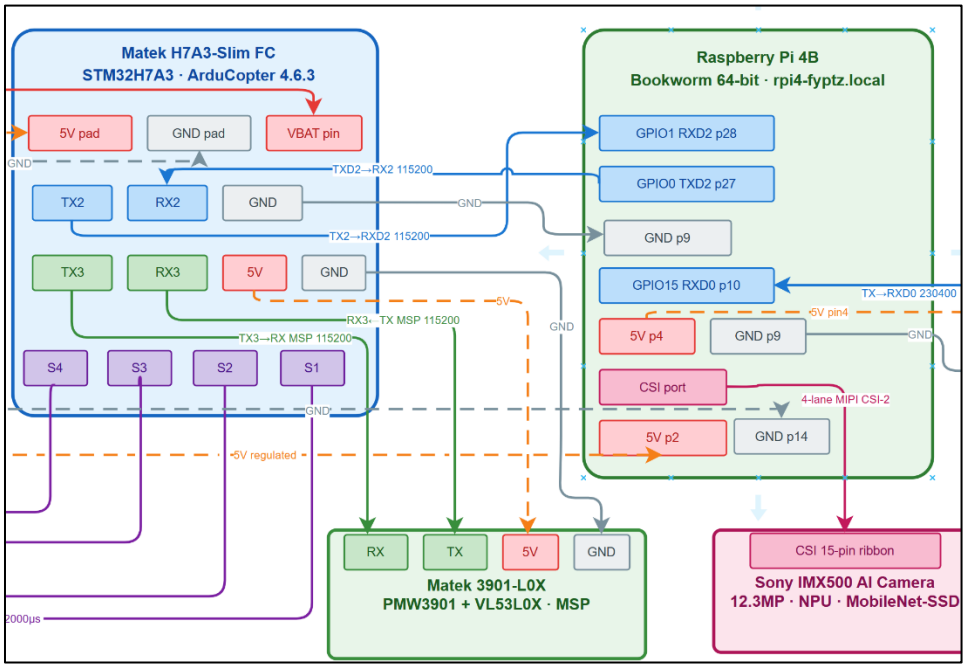


Figure 8-4: 3901-L0X wiring diagram

8.5 LD06 LiDAR

The LD06 uses a rotating prism mirror driven by an internal brushless motor at approximately 2300 RPM. Distance is computed from the phase shift of the reflected 905nm laser signal, and each full rotation produces a 360-degree scan delivered in approximately 30 serial packets covering 12-degree arcs each [5]. At 230400 baud the sensor outputs a complete rotation approximately every 100ms, giving a scan rate of approximately 10Hz.

Real measured data from three test sessions on /dev/ttyAMA0: 10,199 to 104,435 valid measurement points per session, distance range 50mm to 9,992mm, mean confidence 219.8 to 201.2 out of 255. Figure 8-5 shows a real room scan output. The LD06 at approximately 97g is the heaviest non-battery component on the airframe. Its scan plane must be clear of propellers, ESC wires, and internal components. This was the direct cause of a blocked SAR mission in Session 4, documented in Section 13.3.

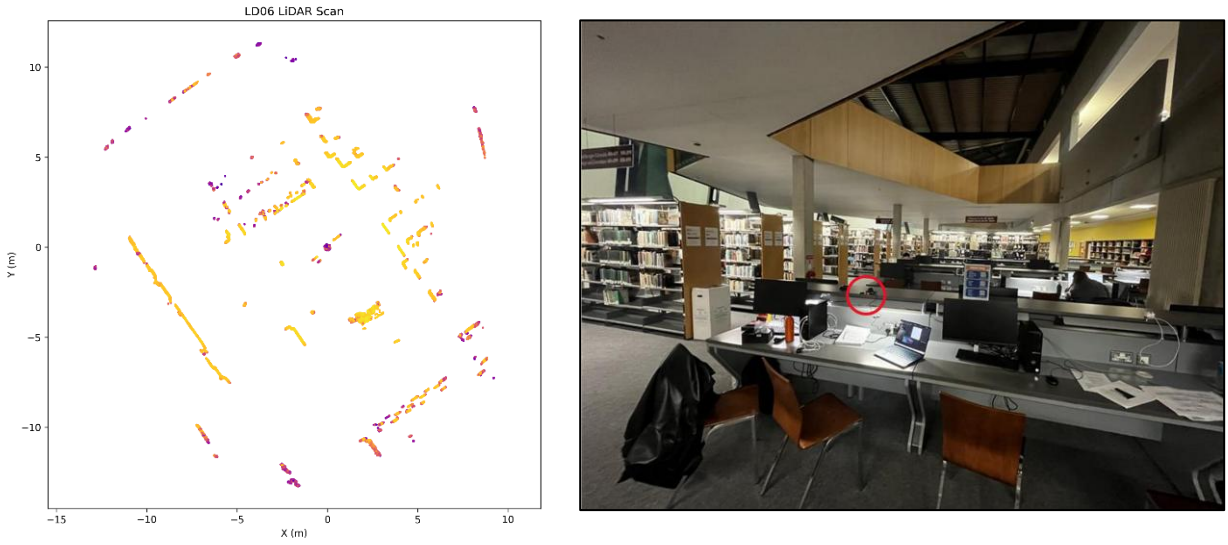


Figure 8-5: LD06 scan output in ATU Library

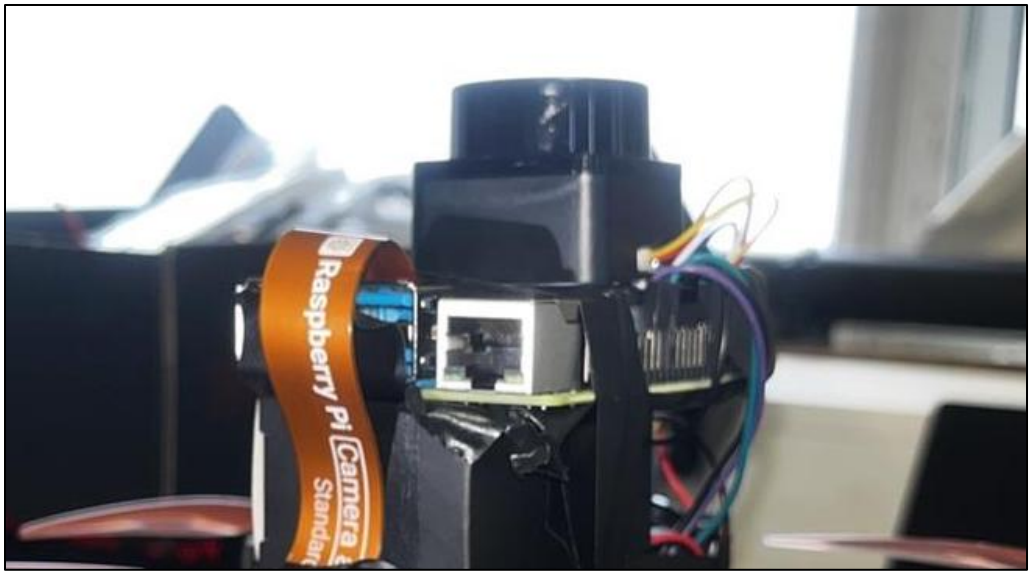


Figure 8-6: LD06 LiDAR [5] mounted on AeroDrop

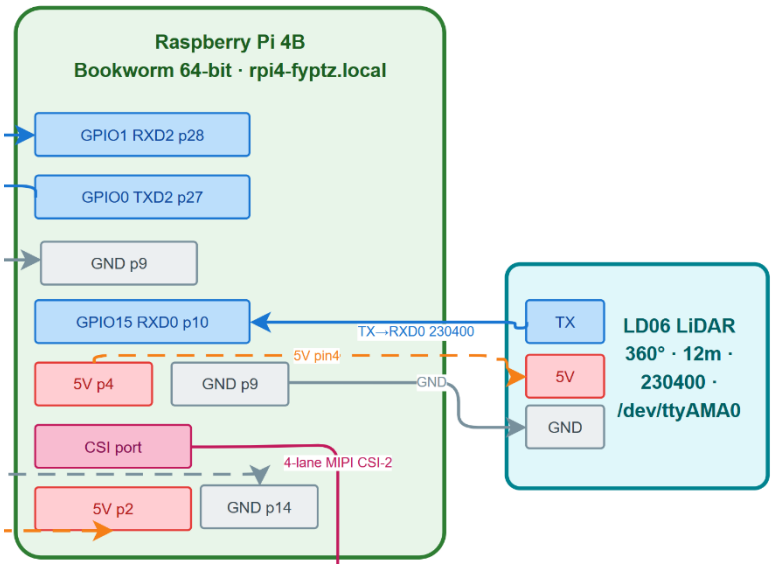


Figure 8-9: LD06 LiDAR Wiring Connections

8.6 Propulsion and the S1 Pad Fault

The EcoIII 2306 2400KV motors on a 4S battery nominal (14.8V) spin at approximately 35,400 RPM unloaded [27]. The FVT LittleBee Summer 35A ESCs use BLHeli32 firmware [28] with conventional PWM at 1000-2000 microsecond throttle range, configured by MOT_PWM_TYPE=0 in ArduCopter.

During motor bring-up testing, MAV_CMD_DO_MOTOR_TEST was sent via pymavlink to each motor in sequence with throttle_type=1 (percentage), throttle=20%, duration=3s. Motors 1, 2,

and 4 responded correctly with the ESC arming beep sequence followed by spin-up. Motor 3 was probed and confirmed it was receiving power and signal ground but did not spin at any tested throttle value.

A systematic swap diagnostic was performed. ESC3 was physically moved to the S1 pad header and ESC1 was moved to the S3 pad header. If Motor 3 with ESC1 on S3 spun correctly, the S3 pad was functional and ESC3 was the faulty unit; if it still did not spin, the S3 output pad itself was the problem. Motor 3 with ESC1 spun correctly on the S3 pad, confirming ESC3 as the faulty component. A replacement FVT LittleBee Summer 35A ESC was purchased and installed. All four motors are now confirmed spinning simultaneously with propellers attached via the website ARM button under battery power.

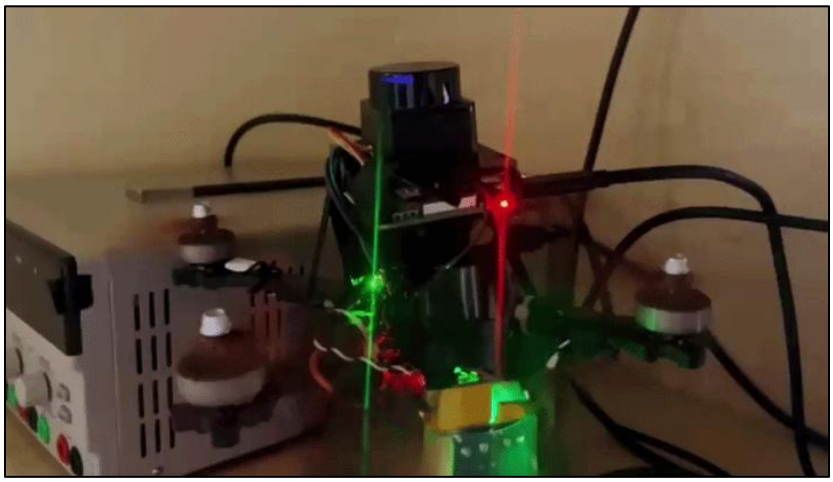


Figure 8-7: All four motors confirmed operational with propellers

8.7 Sony IMX500

The IMX500 is architecturally different from every conventional camera module [8]. Sony built three components on a single stacked die: a 12.3-megapixel CMOS image sensor, a dedicated AI processing chip (NPU), and on-chip SRAM for neural network weights. In conventional camera pipelines, raw pixel data transfers off-sensor to the host CPU which then runs inference. In the IMX500, inference runs on the NPU inside the sensor package and raw pixel data never leaves the sensor. What travels over the CSI interface is structured inference metadata alongside an optional camera frame for display [8].

In AeroDrop, the IMX500 runs MobileNet-SSD [24] using the Picamera2 Python library with the imx500_network_ssd_mobilenetv2_fpnlite_320x320_pp.rpk network package [9]. This model provides 80-class object detection trained on the COCO dataset. Confirmed detections on real hardware (November 2025): laptop at 62% confidence and keyboard at 73% confidence simultaneously in one frame, chair at 62% confidence in a separate session.

A significant obstacle was encountered during IMX500 setup: the Raspberry Pi was initially running Trixie (Debian 13). The rpicam-apps binary on Trixie was compiled without the ENABLE_POST_PROCESSING=1 CMake flag, meaning the IMX500's post-processing pipeline was absent entirely [9]. The symptom was 'No post processing stage found for rpi.imx500'. After multiple attempts to patch repositories and rebuild from source, the correct fix was a complete OS reflash to Bookworm (Debian 12), which provides the Raspberry Pi Foundation's fully supported binary packages [9]. This is a hard OS constraint, documented fully in Section 13.3.

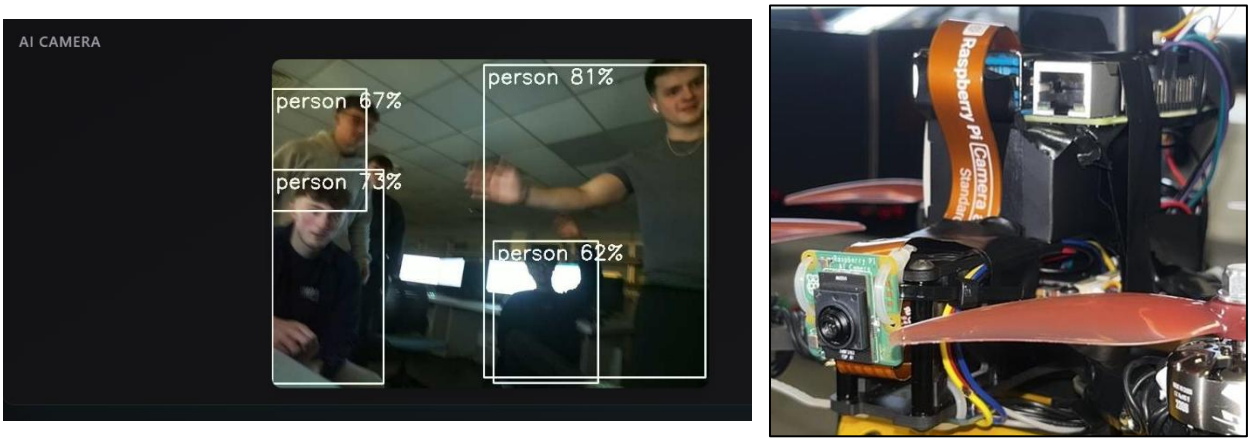


Figure 8-8: Sony IMX500 [8] with live object detection output

8.8 Servo Gripper

The payload delivery mechanism is a Pololu micro servo gripper [39] mounted on the drone chassis. Control is handled entirely from the Raspberry Pi using hardware-timed PWM via the pigpio library, which delegates pulse timing to the VideoCore GPU. This distinction from software PWM is not cosmetic: software PWM on Linux is subject to scheduler jitter under CPU load, which produces erratic pulse widths and corresponding erratic servo behaviour. With pigpio's [38] hardware timing, pulse accuracy is in the microsecond range regardless of companion computer

CPU load, which matters when the same Pi is simultaneously running the LiDAR parser, MAVProxy, and the FastAPI backend.

The servo is wired to GPIO 12 (physical pin 32), with 5V power from pin 2 and ground from pin 14. It operates on the RC PWM convention: a pulse sent every 20ms where pulse width determines angular position. Calibration required empirical tuning. An initial range of 1200 to 1800 microseconds produced only approximately 5mm of mechanical travel, indicating a pulse range too narrow for the servo's full throw. Widening to 600 to 2400 microseconds achieved full travel but produced audible buzzing at the closed position, indicating the servo was straining against its mechanical hard stop. The final values are 500 microseconds (fully open) and 2100 microseconds (fully closed), which achieve full mechanical travel without buzzing. The closed value is deliberately backed off from the 2400 microsecond hard stop by 300 microseconds to eliminate motor strain.

The pigpiod daemon must be running before the backend starts, as the ServoGripper class connects to it at instantiation. If pigpiod is not found, the available flag is set to False and all API endpoints return HTTP 503 gracefully without crashing. The daemon is configured to start automatically on boot via systemd.

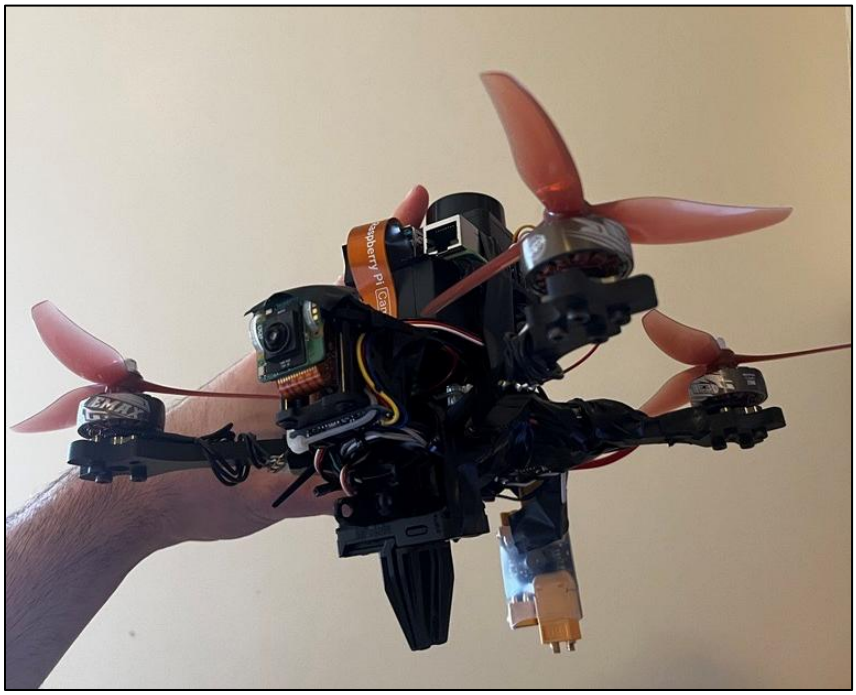


Figure 8-9: Servo Gripper on AeroDrop

9 Firmware Configuration

9.1 ArduCopter Parameter Configuration

ArduCopter's entire runtime behavior is controlled through a flat key-value parameter store of approximately 1,200 parameters [1]. Parameters persist in flash memory across reboots and can be read or written over MAVLink using PARAM_REQUEST_READ, PARAM_REQUEST_LIST, and PARAM_SET messages. This means the complete configuration that produces a working GPS-denied optical flow setup can be saved as a .param file, restored, and version controlled. The firmware itself never needs to change. The full parameter file is provided in Appendix B.

For GPS-denied optical flow, parameters fall into four groups: EKF3 source selection (which sensor feeds which state), sensor protocol assignment (how each sensor communicates with the FC), arming behaviour, and serial port configuration. Table 9-1 lists every critical parameter with its value and the reason its default breaks GPS-denied operation. The most consequential of these, and the one that required the most debugging to understand, is described in depth in Section 9.2. The entire parameter list can be found in Appendix C.

Parameter	Value	Reason default fails GPS-denied operation
FLOW_TYPE	7	Default 0 = disabled. Setting 7 selects MSP protocol. Without this change, the PMW3901 is invisible to ArduCopter regardless of wiring [3].
EK3_SRC1_VELXY	5	Default 3 = GPS. Setting 5 routes optical flow into EKF3's horizontal velocity state. This is the parameter that makes GPS-denied flight possible [2].
EK3_SRC1_VELZ	0	Default 3. No vertical velocity source is available; altitude comes from rangefinder or barometer only.
EK3_SRC1_POSXY	0	Default 3 = GPS. Optical flow gives velocity, not absolute position; EKF3 integrates velocity to estimate position [2].
EK3_SRC1_POSZ	2	Default 1 = barometer. Setting 2 selects rangefinder. The VL53L0X gives centimetre-level accuracy compared to the barometer's metre-level [12].

RNGFND1_TYPE	32	Default 0 = disabled. Setting 32 enables the MSP rangefinder, parsed from the same serial stream as the optical flow data [4].
RNGFND1_MAX_CM	400	VL53L0X practical range limit in centimetres. This was found set to 1 during bring-up, causing EKF3 to reject every altitude reading. Full account in Section 13.2.
SERIAL3_PROTOCOL	32	Default 5 = GPS. Setting 32 selects MSP. The port carrying the 3901-L0X data stream must receive MSP, not GPS [4].
SERIAL3_BAUD	115	MSP baud rate (115200).
SERIAL2_PROTOCOL	2	MAVLink v2 on the companion computer UART.
SERIAL2_BAUD	115	115200 baud, matching the RPi UART dtoverlay configuration.
GPS_TYPE	0	Default 1 = auto-detect. With no GPS module fitted, auto-detect causes boot delays and EKF3 confusion.
COMPASS_USE	0	Motor and ESC cable magnetic fields corrupt compass readings in an indoor environment.
INITIAL_MODE	0	Stabilize mode on boot. GUIDED mode requires a valid EKF3 position estimate before accepting an arm command [1].
ARMING_CHECK	0	Disables the software pre-arm checklist. EKF3's own internal health check remains active regardless, as described in Section 9.2.
MOT_PWM_TYPE	0	Standard PWM. BLHeli32 ESCs are calibrated to the 1000-2000us range [28].
FS_THR_ENABLE	0	RC throttle failsafe disabled. With no RC transmitter connected, this parameter triggered autonomous disarm approximately 2 seconds after every arm. Full account in Section 13.2.

Table 9-1: Critical ArduCopter Parameters for GPS-Denied Optical Flow Operation

9.2 The EKF3 Arming Dependency Chain

The most technically nuanced aspect of the firmware configuration is the relationship between ARMING_CHECK, EKF3, and the sensor chain. ARMING_CHECK=0 disables the software pre-arm checklist. It does not touch EKF3. The filter maintains its own internal health state through a

completely separate code path and will refuse to allow arming if its navigation state has not been initialized [2]. All four conditions below must hold simultaneously before EKF3 sets its health flag:

- Rangefinder healthy: the VL53L0X must report altitude within RNGFND1_MIN_CM and RNGFND1_MAX_CM. The sensor must be powered, which requires LiPo — not USB alone.
- Flow quality sufficient: the PMW3901 must report FLOW_QUALITY above EKF3's internal threshold. The confirmed value on this hardware is 43 out of 255 maximum, representing the number of valid feature correspondences the correlation ASIC found between frames [13].
- Altitude-to-flow coupling valid: EKF3 converts pixel displacement to velocity as velocity = pixel_displacement x altitude / (focal_length_equiv x dt). If altitude is zero or out of range, the result is undefined and the measurement is rejected, as introduced in Section 5.5.
- EKF convergence time elapsed: approximately 15-30 seconds after LiPo power-on are required for the filter to average gyroscope bias and settle accelerometer noise.

This chain was discovered empirically. Covering the PMW3901 to reduce FLOW_QUALITY to 0 caused arm rejection with ARMING_CHECK=0 set and every other software check disabled. The EKF rejection is categorical, and Mission Planner's error message does not always explicitly identify which condition failed. This required systematic debugging rather than simply reading documentation and is described further as a challenge in Section 13.2.

9.3 UART Assignments and the Ground Wire Finding

SERIAL2 (TX2/RX2 pads on the H7A3-Slim) is the companion computer MAVLink link, wired to the Raspberry Pi's GPIO0/1 pins (/dev/ttyAMA2, enabled by dtoverlay=uart2 in /boot/firmware/config.txt at 115200 baud). SERIAL3 (TX3/RX3) carries the optical flow MSP stream from the 3901-L0X. The LiDAR connects to /dev/ttyAMA0 on the Raspberry Pi, with no connection to the FC. The wiring diagram for all UART assignments is provided in Appendix B.

A finding during UART debugging: a loose or missing ground wire on the SERIAL2 FC-to-RPi cable does not produce corrupted data or framing errors. It produces complete silence, with no bytes

received, no errors, and no indication of what is wrong. The UART physical layer requires a shared reference voltage between transmitter and receiver. Without it, the receiver sees only floating signals. This was initially interpreted as a baud rate mismatch or MAVProxy configuration error before the ground wire was found and secured.

10 Software Architecture

10.1 Stack Overview and SIM_MODE

The software stack runs entirely on the Raspberry Pi 4 and is controlled by a single environment variable: SIM_MODE (default: true). When SIM_MODE=false, DroneController, StateMachine, and the drone router are loaded and a background task establishes the MAVSDK connection. The LiDAR always starts regardless of SIM_MODE. This single flag is the only difference between a simulation deployment and a hardware deployment; every other code path is identical. It enabled the full software stack to be developed, tested, and debugged against ArduCopter SITL before any real hardware was powered, with the certainty that deploying to real hardware required only setting one environment variable. The SIM_MODE branching in main.py is examined in Section 11.1.

10.2 DroneController and TelemetrySnapshot

DroneController opens two simultaneous connections to ArduCopter via MAVProxy: MAVSDK on UDP 14552 for telemetry subscriptions and flight commands, and pymavlink on UDP 14553 for 10Hz NED setpoints. Seven background asyncio tasks keep TelemetrySnapshot current, as listed in Table 6-2. The snapshot is a Python dataclass containing: connected, armed, flight_mode, lat, lon, abs_alt, rel_alt, battery_pct, local_north, local_east, local_down, and heading_deg.

A subtle bug discovered during development illustrates how asyncio generators behave: _poll_heading was originally written after the async for loop in _poll_local_position. An asyncio async for loop over a MAVSDK telemetry generator runs indefinitely; the generator never terminates while the connection is live. Any code written after the loop never executes. The result was heading_deg always reading 0.0 in TelemetrySnapshot, causing the occupancy grid to rotate all LiDAR scans by 0 degrees regardless of the drone's actual orientation, producing a systematically incorrect map with no error, no exception, and no indication of the cause. Fixed by splitting _poll_heading into a separate asyncio task. This is examined with code in Section 11.1.

10.3 Arm and Takeoff Implementation

The takeoff implementation evolved across sessions. Early attempts using MAV_CMD_NAV_TAKEOFF via pymavlink returned MAV_RESULT_UNSUPPORTED (result=4) on ArduCopter. This command is not implemented in ArduCopter's MAVLink handler [1]. A subsequent NED setpoint approach, switching to GUIDED mode then immediately sending position setpoints with a negative down value from the ground, reached approximately 1.0m in Session 4. However, in Session 6 with all four motors functional and EKF3 in relative aiding mode, the same approach caused the motors to spin at idle without climbing. GUIDED mode on the ground does not reliably ramp throttle from zero for the ground-to-air transition; ArduCopter requires the NAV_TAKEOFF command to initiate this sequence correctly.

The current implementation uses MAVSDK's action.set_takeoff_altitude() and action.takeoff(). Before issuing the takeoff command, _wait_for_local_position() polls MAVSDK's health stream until is_local_position_ok is True, with a 20-second timeout that raises a descriptive RuntimeError if it expires [7]. After takeoff is issued, the code polls rel_alt until it reaches 85% of the target altitude or a 30-second timeout triggers. The code is examined in Section 11.2.

10.4 Seven-State Flight FSM

StateMachine implements a seven-state FSM with an explicit transition table stored as a Python dict. transition(to) is guarded by asyncio.Lock so only one transition can be evaluated at a time, returning True if valid and applied, False if invalid. trigger_emergency() bypasses the transition table and always succeeds from any state; a safety function must be unconditional. Table 10-1 shows the valid transitions, and the FSM code is examined in Section 11.2.

State	Valid Transitions
IDLE	ARMED
ARMED	TAKEOFF, IDLE, EMERGENCY
TAKEOFF	ARMED, MISSION, RTL, EMERGENCY
MISSION	RTL, EMERGENCY
RTL	LANDED, EMERGENCY
LANDED	IDLE, EMERGENCY

EMERGENCY	IDLE

Table 10-1: FSM State Transition Table

is_airborne() returns True for TAKEOFF, MISSION, and RTL. This is used by the auto-disarm safety watcher: if is_airborne() is True and armed is False (meaning the flight controller disarmed autonomously due to battery failsafe or hard landing), trigger_emergency() is called and the offboard executor is cancelled. The watcher polls at 0.5Hz with a 3-second startup delay to prevent false triggers during the boot sequence.

10.5 OffboardExecutor: 10Hz NED Setpoints

OffboardExecutor sends SET_POSITION_TARGET_LOCAL_NED via pymavlink at 10Hz. The type_mask field in this message is a 16-bit field where each bit tells ArduCopter whether to ignore the corresponding field, as defined in the MAVLink message specification [6]. The value used is 0b0000_1111_1111_1000 (0x0FF8): bits 0-2 cleared (use position: north, east, down), bits 3-11 set (ignore velocity, acceleration, yaw rate), bit 10 cleared (use yaw). Getting even one bit wrong causes the FC to either ignore the message or interpret it as a velocity command, resulting in uncontrolled acceleration. The mask is examined with code in Section 11.3.

Three constants govern executor behavior: OBSTACLE_STOP_M=0.25m (stop if obstacle in forward arc is closer than this), OBSTACLE_ARC_DEG=60 (check plus and minus 60 degrees from forward, a 120-degree total cone), and WAYPOINT_RADIUS_M=0.30m (advance to next waypoint when within 30cm). The obstacle check operates entirely in body frame. The original implementation transformed LiDAR points to world frame before checking proximity but used an incorrect sign convention, causing points behind the drone to appear in the forward arc. This is documented as a challenge in Section 13.3, and the corrected body-frame code is shown in Section 11.3.

10.6 TaskExecutor: Boustrophedon Scan

The boustrophedon scan pattern minimizes dead-heading (travel between rows is minimized by ending each row adjacent to the start of the next). ROW_SPACING_M=0.8m means each sweep

row overlaps with the adjacent row's LiDAR coverage. Table 10-2 shows the generated waypoints for the default mission parameters visible in the dashboard screenshot.

Waypoint	North (m)
WP 1 (row 0, going North)	3.0
WP 2 (row 1, going South)	0.0
WP 3 (row 2, going North)	3.0
WP 4 (row 3, going South)	0.0
WP 5 (return to origin)	0.0

Table 10-2: Generated Waypoints for Default Mission Parameters (row_length=3.0m, rows=4, altitude=1.2m)

Three asyncio coroutines run in parallel inside _run() via asyncio.gather():

1. offboard_executor.run(waypoints) executes the sweep.
2. _monitor_detections() polls the camera pipeline at 5Hz and stamps detections with NED coordinates from TelemetrySnapshot.
3. _update_map() feeds LiDAR snapshots into the OccupancyGrid at 5Hz.

The build_sweep_waypoints() function is examined with code in Section 11.4.

10.7 Occupancy Grid and Bresenham Ray-Cast

The occupancy grid is a 200x200 cell array covering 20m by 20m at 0.10m per cell, centred on the takeoff point. Cell states are FREE (0), OCCUPIED (1), and UNKNOWN (2). For each LiDAR packet, update() rotates the body-frame scan to world frame using the current heading from TelemetrySnapshot, then applies Bresenham's line algorithm [14] from the drone's grid cell to each hit cell, marking every intermediate cell as FREE. The hit cell is marked OCCUPIED. The Bresenham ray-cast distinguishes confirmed-clear space from unobserved space: without it, only hit points are recorded and all space between drone and obstacle remains UNKNOWN even though the LiDAR beam confirmed it is clear. The body-to-world rotation and Bresenham implementation are examined with code in Section 11.5.

The grid serialises to a flat 40,000-element list plus drone position, heading, and detection markers, pushed at 2Hz over the /ws/map WebSocket to the dashboard. The OccupancyMap React component renders green (FREE), red (OCCUPIED), dark (UNKNOWN), yellow ring (detections), and cyan arrow (drone position and heading).

Figure 10-1: Occupancy Grid

10.8 Camera Streamer and Detection Pipeline

The camera streamer uses the Picamera2 Python library [26] combined with the IMX500's inference API. A background thread runs a continuous capture loop: each iteration calls picam2.capture_request() to get a camera frame, reads Picamera2 metadata via req.get_metadata(), and calls imx500.get_outputs(metadata, add_batch=True) to extract the raw neural network outputs from the sensor's NPU. These outputs, bounding boxes, scores, and classes from the MobileNet-SSD model [24], are decoded into detection objects, filtered by confidence threshold 0.3 for display and 0.5 for logging, and stored as latest_detection. OpenCV [22] draws bounding boxes on the frame before JPEG encoding.

This architecture replaced an earlier rpicam-vid subprocess approach. The subprocess version mixed MJPEG binary data and JSON metadata on the same stdout stream, making reliable metadata parsing impossible, as documented in Section 13.3. When task executor's _monitor_detections() finds a detection matching the target class above 0.5 confidence, it reads the current NED position from TelemetrySnapshot, reads the latest_frame JPEG bytes, and writes a Detection dataclass to DetectionLog containing: object class, confidence, north/east coordinates, unix timestamp, and the raw JPEG bytes of the frame at that moment. The full camera pipeline is examined with code in Section 11.6.

10.9 FastAPI Backend and React Dashboard

The FastAPI backend [11] serves the compiled React/Vite production build as static files at http://172.20.10.2:8000. The production build is mandatory: the Vite development server timed

out at 0 bytes received in 12.29 seconds on the first HTTP response under operational RPi load, due to Vite's on-demand JSX transformation competing with MAVProxy, MAVSDK's gRPC subprocess, uvicorn, and the LiDAR parser for CPU time. The production build produces a 63.74 kB gzip JS bundle served at 31ms first response. This issue is documented in Section 13.3.

The dashboard is a two-screen application. Step 1 (Mission Setup) presents a search target input, row length slider (1-10m), number of rows stepper, and cruise altitude slider (0.5-3.0m), with computed coverage and waypoint count displayed in real time. Step 2 (Live Data) uses a six-column CSS grid with the telemetry panel, camera feed with live detections, LiDAR point cloud canvas (scroll-zoom 0.2x to 5x), occupancy map canvas, and a detections list showing class, confidence, and NED position for each confirmed detection. The Export PDF button triggers client-side jsPDF [25] generation with the detection table and occupancy map image.

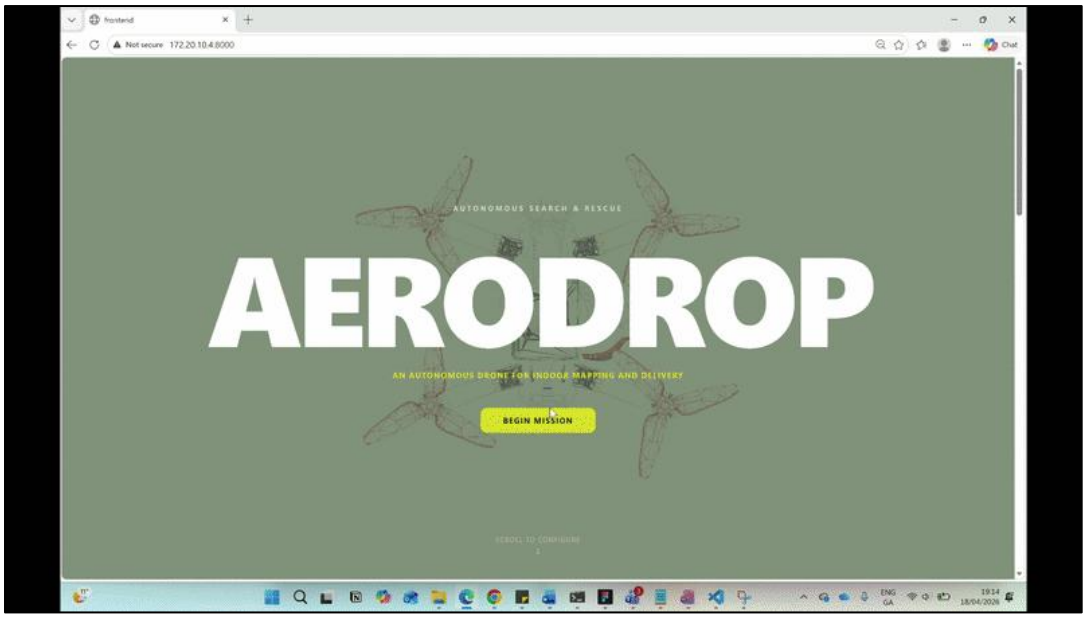


Figure 10-2: Full Website Dashboard

10.10 MAVProxy Bridge

MAVProxy runs as a persistent process on the Raspberry Pi, connecting to the flight controller via /dev/ttyAMA2 at 115200 baud and forwarding the MAVLink stream to UDP 14552 (MAVSDK) and UDP 14553 (pymavlink). A critical operational finding: multiple stale MAVProxy daemon processes accumulate if not explicitly killed before restarting. Each start.sh invocation added a new MAVProxy instance without terminating the previous one, and multiple daemons compete

for the serial port. The one that wins the port gets FC data; all others receive nothing silently, causing intermittent and difficult-to-diagnose connection failures. The fix was adding pkill -f mavproxy.py at startup, followed by a UDP port-readiness poll before launching uvicorn. The full start.sh logic is examined in Section 11.1.

10.11 Servo Gripper Iintegration

The ServoGripper class in backend/app/servo_gripper.py provides hardware-timed PWM control of the payload release mechanism. Three operations are exposed: open() and close() send immediate pulse width commands, and drop() is an async coroutine that opens the gripper, holds for two seconds, then closes. Drop is non-reentrant via asyncio.Lock, meaning a second drop call while one is already executing is ignored rather than queued. A five-second per-class cooldown in the detection monitor prevents the gripper from firing repeatedly on the same stationary target.

Auto-trigger integrates into TaskExecutor's _monitor_detections() coroutine. When a detection matching the configured target class arrives with confidence above 0.5 and the cooldown is clear, gripper.drop() is launched as a background asyncio task so it does not block the detection monitoring loop. The same logic runs in demo_router._live_scan() for ground-test operation without a flying mission.

Manual control is available via three REST endpoints: POST /api/drone/gripper/open, /gripper/close, and /gripper/drop. Corresponding buttons appear in the dashboard above the emergency stop. The fire-and-forget pattern on /gripper/drop returns immediately while the two-second hold executes in the background.

A bug was found during integration: task_executor._monitor_detections() read obj.get("label") and obj.get("confidence"), but camera_streamer._parse() stores detection objects with keys "class" and "score". This key mismatch meant the detection monitor never matched any target class, so no detections were logged, no gripper trigger occurred, and no occupancy map marks were placed. The fix adds fallback key lookup: obj.get("class", obj.get("label", "")) and obj.get("score", obj.get("confidence", 0.0)). This class of bug is invisible at runtime unless you know what output to look for: the detection monitor simply processes zero detections silently.

11 Code Walkthrough

This section traces the execution path from system startup through a complete mission cycle, examining the most significant code decisions in detail. The intent is not to describe every file but to show why the critical parts are written the way they are, and what would happen if they were written differently.

11.1 System Startup and the SIM_MODE Branch (main.py, start.sh)

main.py reads the SIM_MODE environment variable on startup. When SIM_MODE=false, DroneController, StateMachine, and the drone router are imported and dependency injection is performed via set_dependencies(ctrl, sm, lidar). The LiDAR starts unconditionally in the startup event handler. This structure allowed SITL-first development possible: the exact same codebase that ran in simulation runs on real hardware with one environment variable changed.

```
14    SIM_MODE = os.getenv("SIM_MODE", "true").lower() != "false"
15    print(f"[main] SIM_MODE={SIM_MODE}")
```

```
40  # === Drone controller (hardware only) ===
41  if not SIM_MODE:
42      from backend.drone.controller import DroneController
43      from backend.drone.state_machine import StateMachine
44      from backend.drone.router import router as drone_router, ws_router, set_dependencies
45
46      ctrl = DroneController()
47      sm   = StateMachine()
48      set_dependencies(ctrl, sm, lidar)
49      app.include_router(drone_router)
50      app.include_router(ws_router)
51
52
53  @app.on_event("startup")
54  async def startup():
55      lidar.start()
56      if not SIM_MODE:
57          asyncio.create_task(_connect_drone())
```

Source: backend/main.py — SIM_MODE branch and startup event handler

The start.sh script manages process lifecycle. It kills stale MAVProxy and uvicorn processes, starts MAVProxy as a daemon with output directed to 127.0.0.1 rather than 0.0.0.0 (which was found to be unreliable on Linux loopback), then polls for a listening socket on port 14552 for up to 15

seconds before launching uvicorn. This prevents the race condition where uvicorn starts before MAVProxy has begun forwarding packets.

```
    pkill -f "mavproxy.py" 2>/dev/null; sleep 1
    pkill -f "uvicorn backend.main" 2>/dev/null; sleep 0.5

    echo "[AeroDrop] Starting MAVProxy..."
    mavproxy.py --master=/dev/ttyAMA2 --baudrate=115200 \
      --out=udpout:127.0.0.1:14552 \
      --out=udpout:127.0.0.1:14553 \
      --daemon

# Wait until MAVProxy is actually forwarding on UDP 14552 (up to 15s)
echo "[AeroDrop] Waiting for MAVProxy UDP..."
for i in $(seq 1 15); do
  ss -ulnp | grep -q ":14552" && break
  sleep 1
  echo "  ...waiting ($i)"
done
```

Source: start.sh — process management and MAVProxy startup with port-readiness poll

DroneController.connect() opens the MAVSDK connection, waits for the first connected=True state from connection_state(), opens the pymavlink connection in a thread executor (so the blocking wait_heartbeat() call does not freeze the asyncio event loop), and finally launches all seven telemetry polling tasks as asyncio Tasks. The reason _poll_heading is a separate task from _poll_local_position is that an asyncio async for loop over a MAVSDK telemetry generator never returns while the connection is live. When _poll_heading was inside _poll_local_position, it was placed after such a loop and therefore never executed. Two generators must be two tasks.

11.2 FSM Transitions and Takeoff (state_machine.py, controller.py)

The StateMachine class stores allowed transitions as a Python dict mapping each State enum value to a set of reachable states. This makes the guard logic trivially readable and avoids any implicit case-by-case logic. The transition() method acquires asyncio.Lock before evaluating the

guard, preventing concurrent transition attempts from corrupting state. trigger_emergency() acquires the same lock but skips the table check entirely because it must succeed from any state.

```python
# Valid transitions: state -> set of states reachable from it
TRANSITIONS = {
    State.IDLE:      {State.ARMED},
    State.ARMED:     {State.TAKEOFF, State.IDLE, State.EMERGENCY},
    State.TAKEOFF:   {State.ARMED, State.MISSION, State.RTL, State.EMERGENCY},
    State.MISSION:   {State.RTL, State.EMERGENCY},
    State.RTL:       {State.LANDED, State.EMERGENCY},
    State.LANDED:    {State.IDLE, State.EMERGENCY},
    State.EMERGENCY: {State.IDLE},
}
```

```python
    async def transition(self, to: State) -> bool:
        async with self._lock:
            if to in TRANSITIONS.get(self._state, set()):
                self._state = to
                return True
            return False

    async def trigger_emergency(self):
        async with self._lock:
            self._state = State.EMERGENCY
```

Source: backend/drone/state_machine.py — TRANSITIONS dict, transition(), trigger_emergency()

The takeoff sequence in controller.py shows the EKF3 gating in practice. _wait_for_local_position() subscribes to MAVSDK's health stream [7] and blocks asynchronously until is_local_position_ok is True, with a 20-second timeout via asyncio.wait_for(). The error message on timeout names the likely hardware cause directly, which is valuable when debugging on a bench without a display. Only after the gate passes does the code proceed to set the altitude and issue the takeoff command.

```python
    async def _wait_for_local_position(self, timeout: float = 20.0):
        """Block until EKF reports a valid local position estimate, or raise on timeout."""
        async def _check():
            async for health in self._drone.telemetry.health():
                if health.is_local_position_ok:
                    return
        try:
            await asyncio.wait_for(_check(), timeout=timeout)
        except asyncio.TimeoutError:
            raise RuntimeError(
                f"Takeoff aborted — no local position estimate after {timeout:.0f} s. "
                "Check optical flow sensor and rangefinder are connected and producing data."
            )
```

```python
    async def takeoff(self, alt: float = 1.5):
        """Wait for EKF position, then use MAVSDK takeoff (NAV_TAKEOFF) to climb."""
        print("[DroneController] Waiting for EKF local position estimate...")
        await self._wait_for_local_position(timeout=20.0)
        print(f"[DroneController] Position OK — taking off to {alt} m")

        await self._drone.action.set_takeoff_altitude(alt)
        await self._drone.action.takeoff()

        deadline = asyncio.get_running_loop().time() + 30.0
        while asyncio.get_running_loop().time() < deadline:
            await asyncio.sleep(0.5)
            if self.snapshot().rel_alt >= alt * 0.85:
                return
        raise RuntimeError(f"Takeoff timed out — reached {self.snapshot().rel_alt:.2f} m of {alt:.1f} m")
```

Source: backend/drone/controller.py — _wait_for_local_position() and takeoff()

11.3 Offboard Executor: NED Setpoints and Obstacle Check (offboard_executor.py)

The send_ned_setpoint() call uses pymavlink's set_position_target_local_ned_send() with a carefully constructed type_mask. The SET_POSITION_TARGET_LOCAL_NED message definition specifies that each bit in the 16-bit type_mask field indicates whether to ignore the corresponding field [6]. The value 0b0000_1111_1111_1000 (0x0FF8) clears bits 0-2 (use position: north, east, down), sets bits 3-11 (ignore velocity, acceleration, yaw rate), and clears bit 10 (use yaw). Getting even one bit wrong causes the FC to either ignore the message or interpret it as a velocity command, resulting in uncontrolled acceleration.

```python
    def send_ned_setpoint(self, north: float, east: float, down: float, yaw: float = 0.0):
        """Send a position setpoint in body NED frame (metres). Non-blocking."""
        if self._mav is None:
            return
        self._mav.mav.set_position_target_local_ned_send(
            0,          # time_boot_ms (not used)
            1, 1,       # target system, target component
            mavutil.mavlink.MAV_FRAME_LOCAL_NED,
            0b0000_1111_1111_1000,  # position only (ignore vel/accel/yaw-rate)
            north, east, down,
            0, 0, 0,    # velocity
            0, 0, 0,    # acceleration
            yaw, 0,     # yaw, yaw_rate
        )
```

Source: backend/drone/controller.py — send_ned_setpoint() with type_mask breakdown

The obstacle check in offboard_executor.py's run() loop operates entirely in LiDAR body frame. For each scan point, the angle from the sensor's forward direction is computed using atan2(x, y),

where x is right and y is forward in the LD06's body frame convention (0 degrees = forward = Y+). Only points within the 60-degree half-angle forward arc and closer than OBSTACLE_STOP_M=0.25m trigger the blocked state. The reason for body frame rather than world frame is that the original world-frame implementation used an incorrect sign in the rotation matrix, causing points behind the drone to appear in front.

```
                    # Obstacle check — only block for points in the forward arc.
                    # LiDAR body frame: angle 0° = forward = y+, so
                    # point_angle = atan2(x, y). Positive x = right, negative = left.
                    xs, ys, _, _ = self._lidar.snapshot(max_out=200)
                    blocked = False
                    for x, y in zip(xs, ys):
                        dist = math.sqrt(x * x + y * y)
                        if dist > OBSTACLE_STOP_M:
                            continue
                        point_angle_deg = math.degrees(math.atan2(x, y))
                        if abs(point_angle_deg) < OBSTACLE_ARC_DEG:
                            blocked = True
                            break

                    if blocked:
                        self._status.blocked = True
                        await asyncio.sleep(interval)
                        continue
                    self._status.blocked = False
```

Source: backend/drone/offboard_executor.py — body-frame obstacle check in run()

11.4 Task Executor and Boustrophedon Pattern (task_executor.py)

build_sweep_waypoints() generates the boustrophedon waypoint list. The down coordinate is set to negative altitude (NED: negative down = above ground). Even-numbered rows go North (north=row_length, east fixed), odd-numbered rows go South (north=0.0, east fixed). East shifts by ROW_SPACING_M=0.8m between rows. The final waypoint returns to the local origin. The pattern deliberately avoids computing room dimensions in advance; it sweeps the configured row_length and relies on the obstacle check in OffboardExecutor to stop before hitting walls.

```python
    ROW_SPACING_M = 0.8   # metres between sweep rows


def build_sweep_waypoints(
    row_length: float,
    rows: int,
    altitude: float,
) -> List[LocalWaypoint]:
    """
    Generate boustrophedon sweep waypoints in local NED.
    Origin = drone position at mission start.
    Row 0: south end → north end (going North)
    Row 1: north end → south end (going South)
    ...
    """
    down = -altitude   # NED: negative down = positive altitude
    wps: List[LocalWaypoint] = []

    for row in range(rows):
        east = row * ROW_SPACING_M
        if row % 2 == 0:
            # going North
            wps.append(LocalWaypoint(north=row_length, east=east, down=down))
        else:
            # going South
            wps.append(LocalWaypoint(north=0.0, east=east, down=down))

    # Final: return to NED origin column (east=0) at same altitude
    wps.append(LocalWaypoint(north=0.0, east=0.0, down=down))
    return wps
```

Source: backend/drone/task_executor.py — build_sweep_waypoints()

Once waypoints are built, start() offsets each by the current NED position from TelemetrySnapshot so the pattern is centred on wherever the drone currently hovers. It then launches _run() as an asyncio Task. Inside _run(), three coroutines are gathered; if any one returns or raises, asyncio.gather() cancels the others. This is intentional: if the offboard executor finishes the sweep , the detection monitor and map updater should stop at the same moment.

```python
    async def _run(self, waypoints: List[LocalWaypoint], altitude: float):
        # Run offboard + detection monitoring in parallel
        await asyncio.gather(
            self._offboard.run(waypoints),
            self._monitor_detections(),
            self._update_map(),
        )
```

Source: backend/drone/task_executor.py — _run() parallel coroutines via asyncio.gather()

11.5 LiDAR Packet Parsing and Occupancy Grid (lidar_streamer.py, occupancy_grid.py)

The LiDAR driver syncs on the two-byte sequence 0x54, 0x2C before reading each 47-byte packet. This byte-by-byte sync approach handles partial packets and mid-stream starts gracefully. The pyserial serial port is configured at 230400 baud on /dev/ttyAMA0 [29]. The packet format, derived from the LD06 protocol documentation [5], is decoded using Python's struct.unpack():

```
10      MESSAGE_FORMAT = "<xBHH" + "HB" * MEASUREMENT_LENGTH + "HHB"
# fields: length, speed, start_angle, 12x(distance_mm, confidence),
#         stop_angle, timestamp, crc
```

```
69      def _run(self):
70          while not self._stop.is_set():
71              try:
72                  if not self._ser or not self._ser.is_open:
73                      self._open_serial()
74
75                  # sync on 0x54, 0x2C, then read the rest
76                  b = self._ser.read()
77                  if b != b'\x54':
78                      continue
79                  b2 = self._ser.read()
80                  if b2 != b'\x2C':
81                      continue
82                  packet = b'\x54\x2C' + self._ser.read(PACKET_LENGTH - 2)
83                  if len(packet) != PACKET_LENGTH:
84                      continue
```

Source: backend/app/lidar_streamer.py — sync bytes and packet read in _run()

_polar_to_xy() converts each (angle_deg, dist_mm) pair to Cartesian (x, y) in sensor body frame. The LD06's angular zero is forward (Y+), so the conversion uses sin for X (right/left) and cos for Y (forward/back). All distances below 50mm or above 10,000mm are discarded; below 50mm is sensor self-noise and above 10m is out of rated specification [5].

```python
    @staticmethod
    def _polar_to_xy(angle_deg:float, dist_mm:int) -> Tuple[float,float]:
        # meters, sensor at origin. angle 0° forward (Y+), like earlier.
        import math
        a = math.radians(angle_deg)
        r = dist_mm / 1000.0
        x = math.sin(a) * r
        y = math.cos(a) * r
        return x, y
```

Source: backend/app/lidar_streamer.py — _polar_to_xy() body frame coordinate convention

occupancy_grid.py's update() method rotates body-frame points to world frame using the current heading_deg from TelemetrySnapshot. The standard 2D rotation matrix for rotating a body-frame vector (bx, by) by heading angle theta to world NED frame is: world_north = bx*cos(theta) - by*sin(theta) + drone_north; world_east = bx*sin(theta) + by*cos(theta) + drone_east [37].

```python
        heading_rad = math.radians(heading_deg)

        world_hits: List[Tuple[int, int]] = []
        for bx, by in body_frame_points:
            # Rotate body → world frame
            wn = bx * math.cos(heading_rad) - by * math.sin(heading_rad) + drone_north
            we = bx * math.sin(heading_rad) + by * math.cos(heading_rad) + drone_east
            hr, hc = self._ned_to_cell(wn, we)
            if self._in_bounds(hr, hc):
                world_hits.append((hr, hc))
```

Source: backend/app/occupancy_grid.py — body-to-world frame rotation in update()

The Bresenham ray-cast [14] in _bresenham_free() marks all cells between the drone and each obstacle point as FREE. It uses integer arithmetic: the error term err accumulates and determines when to step in the row vs column direction. The endpoint (the obstacle hit cell) is intentionally excluded from the FREE marking and instead set to OCCUPIED by the caller. This is the key to the map's utility: after a full sweep, every LiDAR-observed path is marked FREE and only cells no beam has reached remain UNKNOWN.

```python
    def _bresenham_free(self, r0: int, c0: int, r1: int, c1: int):
        """Mark all cells along the ray from (r0,c0) to (r1,c1) as FREE,
        excluding the endpoint (which is the hit = OCCUPIED)."""
        dr = abs(r1 - r0)
        dc = abs(c1 - c0)
        sr = 1 if r1 > r0 else -1
        sc = 1 if c1 > c0 else -1
        err = dr - dc
        r, c = r0, c0
        while (r, c) != (r1, c1):
            if self._in_bounds(r, c) and self._grid[r][c] != OCCUPIED:
                self._grid[r][c] = FREE
            if r == r1 and c == c1:
                break
            e2 = 2 * err
            if e2 > -dc:
                err -= dc
                r   += sr
            if e2 <  dr:
                err += dr
                c   += sc
```

Source: backend/app/occupancy_grid.py — Bresenham [14] ray-cast _bresenham_free()

11.6 Camera Streamer and IMX500 Detection Pipeline (camera_streamer.py)

The _camera_loop() function initialises Picamera2 [26] with the IMX500 device and configures a 640x480 RGB888 preview at 15fps. Picamera2's RGB888 format stores pixels in BGR order in memory; this matches OpenCV's convention directly, which is why there is no cvtColor call between capture and encode. get_outputs(metadata, add_batch=True) extracts the raw inference results from the IMX500's NPU without any host CPU involvement.

```
                imx500 = IMX500(_MODEL)
                picam2 = Picamera2(imx500.camera_num)

                config = picam2.create_preview_configuration(
                    main={"size": (640, 480), "format": "RGB888"},
                    controls={"FrameRate": 15},
                )
                picam2.configure(config)
                imx500.show_network_fw_progress_bar()
                picam2.start()
                print("[camera_streamer] picamera2 + IMX500 ready", flush=True)

                while True:
                    req = picam2.capture_request()
                    try:
                        frame_bgr = req.make_array("main")  # picamera2 RGB888 = BGR in memory
                        metadata  = req.get_metadata()

                        np_outputs = imx500.get_outputs(metadata, add_batch=True)
                        if np_outputs is not None:
                            objects = _parse(np_outputs)
                            if objects:
                                latest_detection = {"objects": objects, "timestamp": time.time()}

                        for obj in latest_detection.get("objects", []):
                            _draw(frame_bgr, obj)

                        ok, buf = cv2.imencode(".jpg", frame_bgr)
                        if ok:
                            with _frame_lock:
                                latest_frame = buf.tobytes()
                    finally:
                        req.release()
```

Source: backend/app/camera_streamer.py — Picamera2 + IMX500 capture loop

The _parse() function decodes the raw MobileNet-SSD network outputs. The output tensor layout returned by imx500.get_outputs() is: np_outputs[0][0] contains bounding boxes in normalised (y0, x0, y1, x1) format, np_outputs[1][0] contains confidence scores, and np_outputs[2][0] contains class indices [24]. Class indices are mapped to labels using an embedded 80-class COCO label list. Detections below the threshold are discarded before returning.

```python
def _parse(np_outputs, threshold=0.3):
    objects = []
    try:
        boxes   = np_outputs[0][0]   # [N, 4]  y0,x0,y1,x1 normalised
        scores  = np_outputs[1][0]   # [N]
        classes = np_outputs[2][0]   # [N]
        for box, score, cls in zip(boxes, scores, classes):
            score = float(score)
            if score < threshold:
                continue
            y0, x0, y1, x1 = [max(0.0, min(1.0, float(v))) for v in box]
            label = _COCO_LABELS[int(cls)] if int(cls) < len(_COCO_LABELS) else str(int(cls))
            objects.append({
                "class":  label,
                "score":  round(score, 4),
                "x":      round(x0, 4),
                "y":      round(y0, 4),
                "width":  round(x1 - x0, 4),
                "height": round(y1 - y0, 4),
            })
```

Source: backend/app/camera_streamer.py — _parse() decoding MobileNet-SSD [24] output tensors

11.7 REST Endpoints and WebSocket Push: Connecting Frontend to Flight Logic (router.py)

router.py is where the frontend and the flight logic meet. Every button press on the dashboard arrives here as an HTTP POST, and every telemetry update the dashboard displays was pushed from this file over WebSocket. Two patterns are worth examining in detail: the guard logic on flight command endpoints, and the WebSocket telemetry push loop.

Every flight command endpoint follows the same structure. Before touching the controller, it asks the FSM whether the requested transition is valid. If the FSM returns False, the endpoint returns a 409 Conflict immediately; the controller is never called. This means the FSM is the single authority on what is allowed, and the endpoint cannot be tricked into arming a vehicle that is already in MISSION state just because a POST request arrives. The rollback on failure is equally important: if ctrl.arm() raises because the EKF3 health gate was not satisfied, or because MAVProxy was not forwarding, the FSM is rolled back to IDLE. Without the rollback, a failed arm would leave the FSM in ARMED state while the vehicle is actually disarmed, and every subsequent command would operate on a false premise.

```python
@router.post("/arm")
async def arm():
    if not await _sm.transition(State.ARMED):
        return JSONResponse({"error": f"cannot arm from {_sm.state}"}, status_code=409)
    try:
        await _ctrl.arm()
        return {"ok": True, "state": _sm.state}
    except Exception as e:
        await _sm.transition(State.IDLE)
        return JSONResponse({"error": str(e)}, status_code=500)
```

Source: backend/drone/router.py — POST /api/drone/arm with FSM guard and rollback

The /ws/telemetry WebSocket endpoint pushes the full drone state to every connected browser client at 5Hz. It loops forever, serialises a snapshot of TelemetrySnapshot plus the current offboard status and detection list, sends it as JSON, and sleeps for 200ms. The disconnect handler exits the loop cleanly when the browser closes the connection. The _snap_dict() call reads a shallow copy of TelemetrySnapshot, as described in Section 6.3, ensuring the push loop can never read a partially-written state while a polling coroutine is mid-update. The _log.as_dicts() call serialises the detection log without the raw JPEG frame bytes, because sending binary frame data inside a JSON WebSocket message would make the JSON invalid. Frame data is available separately via GET /api/drone/report/json when the operator exports the mission report.

```python
@ws_router.websocket("/ws/telemetry")
async def telemetry_ws(ws: WebSocket):
    from backend.drone.demo_router import get_demo_overrides
    await ws.accept()
    try:
        async for snap in _ctrl.stream_telemetry(hz=5.0):
            os  = _task_exec.offboard_status
            ov  = get_demo_overrides()
            payload = json.dumps({
                "connected":   ov.get("connected",   snap.connected),
                "state":       _sm.state,
                "armed":       ov.get("armed",       snap.armed),
                "flight_mode": ov.get("flight_mode", snap.flight_mode),
                "lat":         snap.lat,
                "lon":         snap.lon,
                "abs_alt":     snap.abs_alt,
                "rel_alt":     ov.get("rel_alt",     snap.rel_alt),
                "battery_pct": snap.battery_pct,
                "local_north": snap.local_north,
                "local_east":  snap.local_east,
                "local_down":  snap.local_down,
                "heading_deg": snap.heading_deg,
                "offboard": {
                    "active":     os.active,
                    "paused":     os.paused,
                    "current_wp": os.current_wp,
                    "total_wp":   os.total_wp,
                    "finished":   os.finished,
                    "blocked":    os.blocked,
                },
                "detections": _log.as_dicts(),
            })
            await ws.send_text(payload)
    except WebSocketDisconnect:
        pass
```

Source: backend/drone/router.py — /ws/telemetry WebSocket 5Hz push loop

One structural detail worth noting is the router prefix arrangement. The drone REST endpoints use prefix='/api/drone' on their APIRouter, but the WebSocket endpoints use a separate ws_router with no prefix. Early testing showed that including the /api/drone prefix in the WebSocket router caused FastAPI to register the path as /api/drone/ws/telemetry rather than /ws/telemetry. The frontend hooks were written expecting /ws/telemetry, so a prefix-free router for WebSocket endpoints was the clean fix rather than updating every client-side URL.

11.8 Servo Gripper: Hardware PWM and Auto-Trigger

ServoGripper connects to the pigpiod daemon via pigpio.pi() at initialisation. All subsequent calls use the set_servo_pulsewidth(pin, microseconds) function, which instructs the VideoCore hardware timer to generate the pulse independently of the CPU. The pulse is sent continuously every 20ms until explicitly changed, which is correct servo behaviour: stopping pulses causes most servos to go limp, while continuous pulse sending holds position.

The drop() coroutine acquires asyncio.Lock before executing. This ensures that if the detection monitor triggers a drop and a manual API request arrives simultaneously, the second request waits rather than interleaving with the hold sequence. The lock is released after the close command, making the gripper immediately available for the next trigger.

The five-second cooldown uses a last_drop_time timestamp compared against time.time(). This operates at the application level rather than inside the servo driver. The consequence is that open() and close() remain individually callable with no cooldown restriction; only the automated drop() sequence is rate-limited.

```
51      async def drop(self):
52          """Open, hold for DROP_HOLD seconds, close. Non-reentrant."""
53          async with self._lock:
54              self.open()
55              await asyncio.sleep(DROP_HOLD)
56              self.close()
57              self._last_deploy = time.time()
```

backend/app/servo_gripper.py

The integration point in _monitor_detections() uses asyncio.create_task() rather than await. The detection monitor polls at 5Hz and must not block on the two-second hold, so the drop is launched as a background task and monitoring continues immediately. If the cooldown check passes, the background task is created; if it fails, nothing happens and the loop continues.

12 Testing and Results

12.1 Testing Methodology

Testing followed a layered approach that matched the development methodology. ArduCopter SITL in WSL2 served as the primary development and validation environment. With SIM_MODE=true and simulated sensor sources, the full software pipeline was exercised before any real hardware interaction. The SITL environment used ArduCopter's actual firmware binary, so every simulation test was directly valid on the real hardware; this was the direct consequence of the PX4-to-ArduPilot SITL migration described in Section 5.1. Hardware testing then proceeded through phases: communication link verification (no motors), sensor subsystem verification (LiPo connected via smoke stopper), motor testing (no propellers), integration testing (full system, no propellers), and finally propelled testing (battery, props, armed).

12.2 MAVLink Communication Link

The SERIAL2 link was verified by confirming HEARTBEAT reception via pymavlink, parameter reads returning correct values (GPS_TYPE=0 confirmed as written), and mode change commands producing confirmed state changes visible in Mission Planner. The MAVSDK smoke test confirmed: FC Link CONNECTED, flight_mode=STABILIZED, heading=24.15 degrees (live IMU reading, confirming non-zero non-default telemetry), and alt=-0.82m (barometric bench reading). The /api/health endpoint returned {status: ok, sim_mode: false}. These confirmed the full data path: FC to SERIAL2 to MAVProxy to UDP 14552 to MAVSDK to DroneController to TelemetrySnapshot to /ws/telemetry to WebSocket to dashboard.

12.3 Optical Flow and EKF3 Verification

The Matek 3901-L0X was verified with LiPo connected and SERIAL3 configured. Mission Planner's Status tab showed: opt_m_x = -0.00011, opt_m_y = 0.02050, opt_qua = 43, confirming the PMW3901 was live and the MSP stream was being parsed correctly by ArduCopter. The EKF3 pre-arm health indicator transitioned to green after approximately 20 seconds of settling time. EKF3 relative aiding was explicitly confirmed in Session 6 via MAVProxy: the message 'AP: EKF3 IMU0 started relative aiding' appeared after setting RNGFND1_MAX_CM=400 and EK3_SRC1_POSZ=2

and rebooting. The rangefinder was verified showing 'distance: 0.13m' decreasing to 0.03m when a hand was moved close to the sensor.

The ARMING_CHECK=0 and EKF3 independence was verified explicitly: with the PMW3901 covered so FLOW_QUALITY dropped to 0, arming was attempted. The arm command was rejected despite ARMING_CHECK=0, confirming the EKF's internal health check is a separate, non-bypassable gate as described in Section 9.2

12.4 LiDAR Verification

The LD06 was connected to /dev/ttyAMA0 via pyserial at 230400 baud and tested with the standalone ld06_viewer.py script, logging raw measurements to CSV. Three sessions produced 197,619 total measurement points across log files of 10,199, 65,401, and 104,435 points. Distance range: 50mm to 9,992mm. Mean confidence: 219.8/255 in the close-range session, 201.2/255 in the longer-range session. The scan image (Figure 8-5) shows clear room structure. The driver was then integrated into the backend and the LidarCanvas React component confirmed live point cloud rendering with visible room geometry.

12.5 IMX500 Camera and Detection Pipeline

The IMX500 pipeline was verified in November 2025, before the flight control stack was built. Confirmed detections on real hardware using MobileNet-SSD via Picamera2: laptop at 62% confidence and keyboard at 73% confidence simultaneously in one frame, chair at 62% confidence in a separate session. All detections rendered as bounding boxes on the dashboard AICamFeed component via OpenCV. Following the rewrite from rpicam-vid subprocess to the Picamera2 API (documented in Section 13.3), detection events were confirmed flowing from the camera pipeline through to the dashboard detection cards and occupancy map detection markers.

12.6 Motor Testing and ESC3 Fault Diagnosis

MAV_CMD_DO_MOTOR_TEST via pymavlink (throttle_type=1, throttle=20%, duration=3s) confirmed Motors 1, 2, and 4 responding correctly. Motor 3 on the S3 pad produced the ESC arming beep confirming power and signal ground, but zero RPM at all tested throttle values. The

swap diagnostic (ESC3 to S1 pad, ESC1 to S3 pad) confirmed Motor 3 spun correctly with ESC1 on S3, proving ESC3 was the faulty unit. A replacement was purchased and installed. All four motors are now confirmed spinning simultaneously with propellers attached via the website ARM button.

12.7 Arm and Takeoff Status

GUIDED mode arm was confirmed via MAVProxy interactive session in Session 6 after identifying and disabling the RC failsafe (FS_THR_ENABLE=0). The website ARM button was then confirmed working under battery power with all four motors spinning. A full takeoff attempt with propellers fitted was cut short by battery depletion under motor load. No vehicle damage occurred. The battery is currently charging. The takeoff implementation has been updated to use MAVSDK's action.takeoff() with EKF3 position validation (Section 10.3), and the first free-flight hover trial is the immediate next step.

12.8 Servo Gripper

The servo gripper was verified functional through both manual and auto-triggered paths. Manual verification used the three dashboard buttons (Open, Drop, Close) to confirm the full mechanical range of motion. The servo opened to the 500 microsecond position and closed to the 2100 microsecond position cleanly on command. The two-second drop hold was confirmed by observation: open, two-second visible hold, close.

Auto-trigger verification was conducted in ground-test demo mode: with the system in demo scan, the IMX500 detected a person at 68% confidence, the five-second cooldown was clear, and the gripper executed a drop automatically without any manual input. The corresponding log line [Demo] AI detection: person 0.68 confirmed the confidence threshold and class match. The detection key mismatch bug (described in Section 13.3.x) was identified during this verification when the auto-trigger initially failed to fire despite confirmed detections on the dashboard.

12.9 Full Pipeline Integration

The SAR pipeline was partially validated with the drone hand-carried without propellers. The system armed, the task executor was started, the occupancy grid began building from real LiDAR data as the drone was moved through the lab space, and detection events were observable on the dashboard. The pipeline was blocked in Session 4 by false obstacle triggers from loose internal wiring within the LiDAR scan plane. Properly mounting all electronics on the chassis before the next test was the direct response. The pipeline components (FSM transitions, offboard executor waypoint sequencing, map building, and detection monitoring) are confirmed functional in SITL and partially validated on real hardware.

13 Challenges and Problem Solving

This section documents the most significant technical challenges encountered during development, the diagnostic process used, and the resolution applied. These represent genuine engineering findings that shaped the final system design.

13.1 Hardware Challenges

13.1.1 PDB BEC Failure and UBEC Voltage Sag Under Load

The PDB-XT60's onboard 5V BEC became unreliable after repeated bench testing cycles, collapsing within a few seconds under any sustained load. An external UBEC was installed as a replacement. A second, separate problem then emerged in Session 6: when all four motors spin up under battery power, the PDB bus voltage sags, the UBEC input follows the sag, and the Raspberry Pi browns out. The temporary fix is powering the RPi via USB-C from a separate source. The permanent fix for flight is a USB power bank on the frame, fully isolated from the motor power bus.

13.1.2 ESC3 Fault: Isolation by Swap Diagnostic

Motor 3 failed to spin. The systematic swap diagnostic (ESC3 to S1 pad, ESC1 to S3 pad) confirmed ESC3 was the faulty unit. A replacement was purchased and installed. All four motors are now operational, as described in Section 12.6.

13.2 Firmware and Configuration Challenges

13.2.1 RNGFND1_MAX_CM Set to 1: Silent EKF3 Failure

RNGFND1_MAX_CM had a value of 1 (1 centimeter) when it should be 400. Every distance reading from the VL53L0X was above 1cm and therefore rejected by EKF3 [2]. The filter never received altitude data, could not compute optical flow velocity from the altitude-scaling formula in Section 5.5, and never initialized position. The symptom was 'Mode change to GUIDED failed: requires position', which appeared identical to a flow sensor configuration problem. Systematic inspection of every EKF3-related parameter via MAVProxy found RNGFND1_MAX_CM=1. Setting it to 400 and rebooting produced 'EKF3 IMU0 started relative aiding' immediately.

13.2.2 RC Throttle Failsafe Without RC Transmitter

After arming via the website with all four motors running, the motors would spin for approximately two seconds then stop. The root cause was FS_THR_ENABLE=1: ArduCopter's RC throttle failsafe. With no RC transmitter connected, ArduCopter detects no RC signal approximately two seconds after arming and triggers the configured failsafe action. Setting FS_THR_ENABLE=0 resolved the issue.

13.2.3 ARMING_CHECK=0 Does Not Bypass EKF3

ARMING_CHECK=0 was expected to allow arming regardless of sensor state. It disables the software pre-arm checklist but does not touch EKF3's independent internal health state [2]. Covering the optical flow sensor to reduce FLOW_QUALITY to 0 caused arm rejection with ARMING_CHECK=0 fully set. This behavior is not described clearly in ArduPilot documentation and was discovered empirically.

13.3 Software Challenges

13.3.1 _poll_heading Deadlock Inside Async Generator

The heading poll was written after the async for loop in _poll_local_position. An asyncio async for loop over a MAVSDK telemetry generator never returns while the connection is live. Code written after it never executes. heading_deg always read 0.0, causing the occupancy grid to rotate all LiDAR scans to the same orientation regardless of the drone's real heading. The map was systematically wrong with no error or exception. Fixed by making _poll_heading an independent asyncio task.

13.3.2 Duplicate WebSocket Decorator and Route Corruption

The /ws/lidar route was registered twice with stacked @app.websocket decorators on the same function. FastAPI silently dropped one registration, corrupting the route table. The dashboard showed DISCONNECTED with no obvious error and no stack trace. Fixed by removing the duplicate decorator.

13.3.3 MAVProxy Stale Daemon Accumulation

Each start.sh invocation added a new MAVProxy daemon without terminating the previous one. Multiple daemons competed for /dev/ttyAMA2; only one could hold the port and the others received nothing silently, causing intermittent and difficult-to-diagnose connection failures. Fixed by adding pkill -f mavproxy.py at startup followed by a UDP port-readiness poll before launching uvicorn.

13.3.4 Obstacle Check Coordinate Frame Bug

The initial obstacle check transformed LiDAR body-frame points to world frame before checking proximity, but the rotation matrix used an incorrect sign convention. Points behind the drone appeared in the forward arc, and the obstacle stop triggered from the drone's own tail regardless of actual forward obstacles. Fixed by performing the check entirely in body frame using the LiDAR's native coordinate convention (0 degrees = forward = Y+).

13.3.5 Camera Metadata Routing: rpicam-vid to Picamera2 Migration

The rpicam-vid subprocess with '--metadata --metadata-format json -o -' routes both MJPEG frames and metadata to stdout, mixing binary with JSON in the same stream. The metadata reader thread was attached to stderr and received nothing; latest_detection stayed empty forever. Fixed by migrating to the Picamera2 Python library [26], which provides programmatic access to both frames and inference results through separate, clean interfaces.

13.3.6 Vite Dev Server Performance Collapse on Raspberry Pi

Vite's on-demand JSX transformation timed out at 12.29 seconds on the first HTTP response under operational RPi load. Fixed by building once with npm run build and serving the static bundle from FastAPI. First response time: 31ms. Production build size: 63.74 kB gzip.

13.3.7 Camera Blue Tint from BGR/RGB Inversion

Picamera2 with RGB888 format stores pixels in BGR order in memory. Applying cv2.COLOR_RGB2BGR treated BGR data as RGB, producing a double conversion and swapping red and blue channels in the output. Fixed by removing the cvtColor call; the raw array from make_array('main') is already in BGR order and correct for OpenCV encoding directly.

13.3.8 FastAPI /assets Static Mount Path Mismatch

Vite generates hashed JS bundles at /assets/index-*.js and references them with absolute root paths in index.html. FastAPI was mounted at /static, so requests for /assets/*.js returned 404. Fixed by adding a second StaticFiles mount at /assets pointing to the build output assets subdirectory.

13.4 Project Management and Operational Challenges

13.4.1 Late Hardware Delivery Compressing Integration and Testing Time

The physical drone components arrived at the end of Semester 2 week 10 of the academic calendar), leaving only two sprint cycles before the live demonstration deadline. Industry best practice for embedded autonomous systems allocates integration and hardware-in-the-loop testing as a dedicated phase of comparable length to software development. With hardware arriving this late, those two activities had to run concurrently with active software development, poster production, and report writing. The result was that full end-to-end flight testing, which requires stable hardware, a tuned flight controller, and a validated software stack simultaneously, was never achievable within the available window. The response was to front-load all software validation in simulation (SITL and HITL) and accept a scoped demonstration: the SAR pipeline running with the drone physically moved by hand rather than flying autonomously. This was a deliberate risk management decision rather than an oversight.

13.4.2 Single-Developer Constraint on a Multi-Discipline System

AeroDrop spans embedded firmware configuration, low-level hardware bring-up, Linux systems, asynchronous Python, computer vision, real-time communication, and a React frontend. In a team project this workload would be distributed across specialisations. As a solo capstone, every diagnostic, every configuration decision, and every implementation fell to one person. This made context-switching expensive: diagnosing an EKF3 failure requires a different mental model than debugging a FastAPI route conflict or a React WebSocket disconnect, and shifting between them in a single session produced a higher error rate and longer resolution times than would be expected in a team environment. The Jira backlog reflects this directly: tasks that would normally

be parallelised across a team of three or four were sequenced into a single-threaded sprint plan across thirteen sprints.

13.4.3 Hardware Fault Cascade Near Demonstration Deadline

Two independent hardware faults emerged in Sprint 11, within days of each other and within two weeks of the live demonstration: ESC3 failed, and the S1 solder pad on the flight controller was found to be faulty, preventing a direct replacement without rework. Neither fault was caused by software error or operator damage; both were component-level failures. Sourcing a compatible replacement ESC (DYS Aria Slim 40A BLHeli32) and a new LiPo battery required lead time that consumed the buffer originally allocated to flight testing. Managing two concurrent hardware procurement issues while maintaining software development momentum and meeting academic deadlines tested the project's contingency margin to its limit.

14 Ethics

Autonomous aerial vehicles operating in indoor environments near people and infrastructure raise ethical considerations that were design inputs for this project, not afterthoughts.

Physical safety is the primary concern. AeroDrop addresses this through multiple independent layers: the FSM prevents arming without EKF3 health confirmation, meaning the vehicle cannot take off if its navigation is not trustworthy, the offboard executor stops on forward obstacles at 0.25m, the emergency stop button sends an immediate trigger_emergency() from any flight state, the auto-disarm watcher catches FC-initiated disarms, and all testing was conducted in a cleared space with no bystanders within the flight envelope.

Data privacy is relevant because the IMX500 camera captures and processes visual information in real time. In search-and-rescue or inspection contexts, this includes potentially sensitive imagery of environments and individuals. AeroDrop stores all detection data, including captured JPEG frames, locally on the Raspberry Pi. No data is transmitted to external servers. The onboard inference model means raw pixel data does not leave the sensor, limiting the scope of what could be intercepted even if the local network connection were compromised.

Accountability in autonomous systems is a broader question. AeroDrop maintains a human operator in the loop for mission initiation and retains a continuously accessible emergency stop. Fully autonomous operation without a human oversight channel is architecturally possible but would require a more formal safety analysis before deployment in any real operational context.

Under Irish and EU law, drone operations are regulated by SI 563 of 2022 implementing EASA framework regulations. AeroDrop operates exclusively indoors in a controlled academic environment and is classified as an experimental prototype. Any deployment outside a controlled environment would require appropriate EASA category authorization based on operational risk classification. The use of AI assistance tools during development is disclosed in the Acknowledgements section in the interest of academic integrity [31,32].

15 Conclusion

AeroDrop demonstrates that a GPS-denied indoor autonomous aerial system is achievable on commercial hardware with open-source firmware, implementing a complete autonomy stack on a companion computer with no RC transmitter and no external localization infrastructure.

The optical flow and ToF altitude sensing chain through EKF3 is confirmed live on real hardware, with relative aiding explicitly verified via the 'EKF3 IMU0 started relative aiding' message. The MAVLink link between the Raspberry Pi and flight controller is stable. The LD06 LiDAR is parsing real scans with 104,435 measured points in a single session. The IMX500 is detecting objects at 62-81% confidence on the dashboard. A systematic ESC swap diagnostic correctly identified ESC3 as the faulty component and all four motors are now confirmed spinning with propellers. Every software module was validated end-to-end in ArduCopter SITL and deployed to real hardware.

The challenges documented in Section 13 each have specific root causes and specific fixes. RNGFND1_MAX_CM=1 silently prevented the EKF from ever accepting altitude data. The RC failsafe was triggering autonomous disarm two seconds after every arm. The UBEC voltage sag under motor load required isolating the Raspberry Pi power rail from the motor bus. The camera metadata routing issue required migrating from a subprocess approach to the Picamera2 API [26]. Each represents a real engineering finding with a traceable path from symptom to root cause to fix.

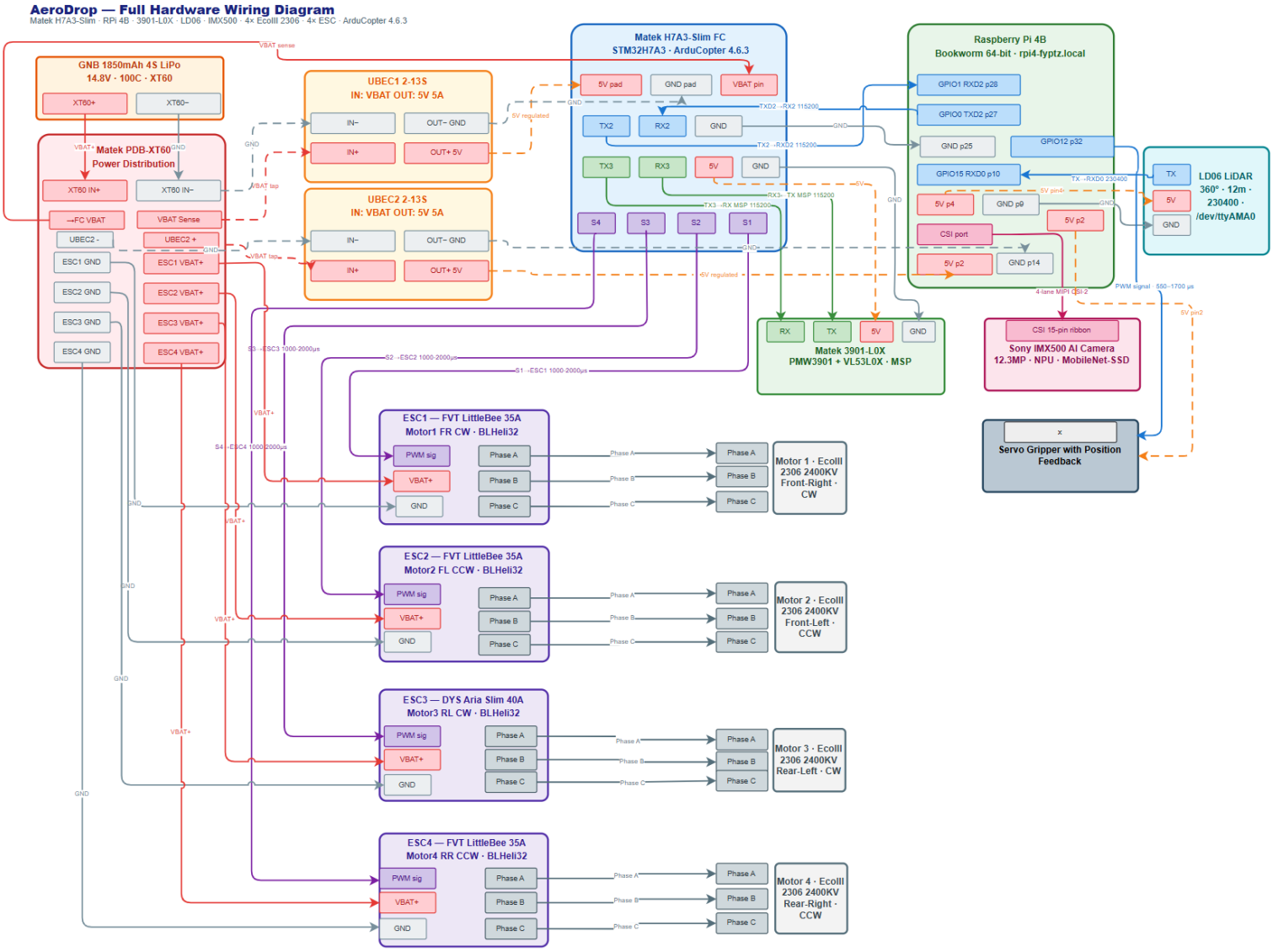
The remaining step to a full autonomous flight demonstration is concrete: one full hover trial on a charged battery to characterize EKF3 position hold stability under real optical flow conditions. All software modules are staged and validated. The servo gripper is integrated and confirmed working through both manual and auto-triggered paths, providing the payload delivery capability that closes the loop from detection to physical response. The SAR pipeline consisting of boustrophedon scan, live occupancy map, IMX500 detection, NED-logged report, gripper drop on target, and PDF export, is staged and ready for first flight.

16 Appendices

Appendix A: Bill Of Materials

Component	Quantity	Unit Cost (€)	Total Cost (€)	Purchased By
Matek H7A3-Slim Flight Controller	1	60.52	60.52	ATU
EcoIII 2306 2400KV Motor	4	51.06	51.06	ATU
FVT LittleBee Summer 35A BLHeli32 ESC	4	51.06	51.06	ATU
DYS Aria Slim 40A BLHeli32 ESC (replacement)	1	38	56	Tamer (Myself)
PDB-XT60 Power Distribution Board	1	15	30	Tamer (Myself)
Matek 3901-L0X Optical Flow + ToF Sensor	1	25.62	25.62	ATU
LD06 LiDAR	1	48.77	48.77	ATU
Raspberry Pi 4	1	59.7	59.7	ATU
IMX500 AI Camera Module	1	59.66	59.66	ATU
LiPo Battery (original)	1	25	25	ATU
LiPo Battery (replacement)	1	27	53	Tamer (Myself)
LiPo Battery Charger	1	22	48	Tamer (Myself)
Drone Frame	1	35	35	ATU
UBEC (external 5V regulator)	3	45	60	Tamer (Myself)
Propellers	4	3	3	ATU
XT60 Connectors / Wiring	2	2.5	5	Tamer (Myself)
Smoke Stopper	1	16	25	Tamer (Myself)
Servo Gripper	1	40	55	Tamer (Myself)
Total Bought by Me: €332		Total By ATU: €420		Total: €752

Appendix B: Wiring and Interconnect Diagram



Appendix C: ArduCopter Full Parameter List

WORKING4MOTORS
.param

Appendix D: Software Module Structure

Module	Purpose
backend/main.py	FastAPI app entry. SIM_MODE flag controls drone module loading. LiDAR always starts.
backend/drone/controller.py	DroneController: MAVSDK + pymavlink dual connection, 7 asyncio telemetry tasks, TelemetrySnapshot, arm/takeoff/land/rtl with EKF3 gating.
backend/drone/state_machine.py	StateMachine: 7-state FSM, TRANSITIONS dict, asyncio.Lock guarded transition(), trigger_emergency().
backend/drone/router.py	All REST endpoints (/arm, /takeoff, /land, /rtl, /emergency, /reset, /task/start, /task/cancel, /task/pause, /task/resume, /report/json) and WebSocket endpoints (/ws/telemetry at 5Hz, /ws/map at 2Hz).
backend/drone/offboard_executor.py	10Hz NED setpoints via pymavlink. OBSTACLE_STOP_M=0.25, OBSTACLE_ARC_DEG=60, WAYPOINT_RADIUS_M=0.30. Body-frame obstacle check. Pause/resume/cancel.
backend/drone/task_executor.py	build_sweep_waypoints() boustrophedon, ROW_SPACING_M=0.8. Three parallel asyncio coroutines: offboard run, detection monitor, map update.
backend/app/lidar_streamer.py	LD06 binary parser via pyserial on /dev/ttyAMA0 at 230400 baud. Daemon thread. Ring buffer. Body-to-Cartesian conversion. snapshot() output.
backend/app/occupancy_grid.py	200x200 grid, 0.10m/cell, 20mx20m. Bresenham ray-cast FREE marking. Heading-based body-to-world rotation. Thread-safe. serialise() flat 40,000 cells.

backend/app/camera_streamer.py	Picamera2 + IMX500 Python API. Daemon thread. MobileNet-SSD output decode. OpenCV bounding boxes. MJPEG HTTP stream. latest_detection and latest_frame globals.
backend/app/detection_log.py	Detection dataclass: class, confidence, north, east, timestamp, frame_jpg. Thread-safe list. as_dicts() excludes frame bytes for WebSocket/JSON.
backend/app/servo_gripper.py	pigpio hardware PWM, open/close/async drop() with 2s hold, asyncio.Lock non-reentrant, 5s cooldown. Graceful degradation if pigpiod unavailable.
frontend/src/App.jsx	Two-screen layout: Mission Setup + FlightControl, Live Data (Telemetry, Camera, LiDAR, OccupancyMap, Detections, Export PDF).
frontend/src/components/OccupancyMap.jsx	Canvas 500x500: green=FREE, red=OCCUPIED, dark=UNKNOWN, yellow ring=detections, cyan arrow=drone. Save PNG.
frontend/src/components/LidarCanvas.jsx	Canvas LiDAR point cloud. Scroll-wheel zoom 0.2x-5x. Nearest distance display.
frontend/src/hooks/useTelemetryStream.js	WebSocket hook with 3s auto-reconnect and 6s stale watchdog. Feeds TelemetryBox.
start.sh	Kills stale MAVProxy/uvicorn. Launches MAVProxy daemon. 15s UDP readiness poll. Launches uvicorn backend.

Appendix E: Code

Github Code: Master branch : https://github.com/TamerZraiq/AutonomousDroneFYP

Appendix F: Video

Youtube Link: https://youtu.be/xwGKnfyeetg

Appendix G: Christmas Demo Slides

Canva Link : https://canva.link/l9367o673ftf76h

Appendix H: Final Demo Slides

Canva Link : https://canva.link/fapaegxadt0hio3

17 References

[1] ArduPilot Development Team. "ArduCopter Documentation." ArduPilot. [Online]. Available: https://ardupilot.org/copter/. [Accessed: 2025].

[2] ArduPilot Development Team. "EKF3 Source Selection and Lane Switching." ArduPilot Docs. [Online]. Available: https://ardupilot.org/copter/docs/common-ek3-sources.html. [Accessed: 2025].

[3] ArduPilot Development Team. "Optical Flow Sensor Setup." ArduPilot Docs. [Online]. Available: https://ardupilot.org/copter/docs/common-optical-flow-sensor-setup.html. [Accessed: 2025].

[4] Matek Systems. "Matek 3901-L0X Optical Flow and LiDAR Sensor." [Online]. Available: http://www.mateksys.com/?portfolio=3901-l0x. [Accessed: 2025].

[5] LD Robot. "LD06 LiDAR Development Manual." [Online]. Available: https://www.ldrobot.com/. [Accessed: 2025].

[6] Meier, L. et al. "MAVLink: Micro Air Vehicle Message Marshalling Library." [Online]. Available: https://mavlink.io/en/. [Accessed: 2025].

[7] MAVSDK Contributors. "MAVSDK-Python API Reference." [Online]. Available: https://mavsdk.mavlink.io/main/en/python/. [Accessed: 2025].

[8] Sony Semiconductor Solutions. "IMX500 Intelligent Vision Sensor." [Online]. Available: https://developer.sony.com/imx500. [Accessed: 2025].

[9] Raspberry Pi Foundation. "Raspberry Pi AI Camera Documentation." [Online]. Available: https://www.raspberrypi.com/documentation/accessories/ai-camera.html. [Accessed: 2025].

[10] Raspberry Pi Foundation. "Raspberry Pi 4 Model B." [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/. [Accessed: 2025].

[11] Ramirez, S. et al. "FastAPI." [Online]. Available: https://fastapi.tiangolo.com. [Accessed: 2025].

[12] STMicroelectronics. "VL53L0X Time-of-Flight Ranging Sensor Datasheet." [Online]. Available: https://www.st.com/en/imaging-and-photonics-solutions/vl53l0x.html. [Accessed: 2025].

[13] PixArt Imaging. "PMW3901MB Optical Motion Tracking Chip Datasheet." Available via sensor vendor documentation. [Accessed: 2025].

[14] Bresenham, J.E. "Algorithm for computer control of a digital plotter." IBM Systems Journal, vol. 4, no. 1, pp. 25-30, 1965.

[15] ArduPilot Development Team. "MAVProxy Documentation." [Online]. Available: https://ardupilot.org/mavproxy/. [Accessed: 2025].

[16] ArduPilot Development Team. "ArduCopter SITL Simulator." [Online]. Available: https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html. [Accessed: 2025].

[17] Python Software Foundation. "asyncio: Asynchronous I/O." Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/asyncio.html. [Accessed: 2025].

[18] Team BlackSheep. "Source One V5 5-inch Frame." [Online]. Available: https://www.team-blacksheep.com/products/prod:source_one_5in. [Accessed: 2025].

[19] FVT Model. "LittleBee Summer 35A ESC BLHeli32." [Online]. Available: https://www.fvtmodel.com/. [Accessed: 2025].

[20] React Contributors. "React Documentation." [Online]. Available: https://react.dev. [Accessed: 2025].

[21] IETF. "RFC 6455: The WebSocket Protocol." [Online]. Available: https://tools.ietf.org/html/rfc6455. [Accessed: 2025].

[22] OpenCV Contributors. "OpenCV Documentation." [Online]. Available: https://docs.opencv.org/. [Accessed: 2025].

[23] Vite Contributors. "Vite Build Tool Documentation." [Online]. Available: https://vitejs.dev. [Accessed: 2025].

[24] Howard, A. et al. "Searching for MobileNetV3." IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[25] jsPDF Contributors. "jsPDF Library." [Online]. Available: https://github.com/parallax/jsPDF. [Accessed: 2025].

[26] Raspberry Pi Foundation. "Picamera2 Library." [Online]. Available: https://github.com/raspberrypi/picamera2. [Accessed: 2025].

[27] EcoIII. "EcoIII 2306 2400KV Brushless Motor Specifications." Available via vendor documentation. [Accessed: 2025].

[28] Bitdump. "BLHeli32 ESC Firmware." [Online]. Available: https://github.com/bitdump/BLHeli. [Accessed: 2025].

[29] pyserial Contributors. "pyserial Documentation." [Online]. Available: https://pyserial.readthedocs.io/. [Accessed: 2025].

[30] Welch, G. and Bishop, G. "An Introduction to the Kalman Filter." University of North Carolina at Chapel Hill, Technical Report TR 95-041, 1995.

[31] Anthropic. "Claude." AI assistant consulted for debugging, code structure suggestions, and technical writing assistance during this project. [Online]. Available: https://www.anthropic.com/claude. [Accessed: 2025-2026].

[32] OpenAI. "ChatGPT." AI assistant consulted for debugging and code suggestions during this project. [Online]. Available: https://openai.com/chatgpt. [Accessed: 2025-2026].

[33] Matek Systems. "H7A3-Slim Flight Controller." [Online]. Available: http://www.mateksys.com/?portfolio=h7a3-slim. [Accessed: 2025].

[34] Harris, C.R. et al. "Array programming with NumPy." Nature, 585, pp. 357-362, 2020.

[35] "ARCHIVED TOPIC — Copter Documentation." *Ardupilot.org*, 2024, ardupilot.org/copter/docs/common-msp-overview.html.

[36] *Coverage path planning: The Boustrophedon cellular …* Available at: https://scispace.com/papers/coverage-path-planning-the-boustrophedon-cellular-43u5b5ntj3

[37] "7.1. Moving in Three Dimensions — Introduction to Robotics and Perception." *Roboticsbook.org*, 2023, www.roboticsbook.org/S71_drone_state.html.

[38] Croston, J. "pigpio — A Python module for the Raspberry Pi which talks to the pigpio daemon." [Online]. Available: https://abyz.me.uk/rpi/pigpio/. [Accessed: 2026].

[39] Pololu Corporation. "Pololu Micro Gripper for Servo Motors." [Online]. Available: https://www.pololu.com/category/167/grippers. [Accessed: 2026].